MACSE502 Programming for Data Science

Syllabus Gap Code

Code for the syllabus topics that the lab-based cheat sheet does not cover, in the same task-table format (T.No | Task Desc | Program code).

Module 1	Python: conditions, data structures, stack / queue / priority queue, strings, exceptions, OOP, files, REST APIs, NLTK, ChatterBot
Module 2	pandas Series and data operations, text data, merging, SciPy, Scrapy, scikit-learn, XGBoost, Keras, TensorFlow, PyTorch, Folium, more plots
Module 3	Scala: functions, variables, strings, numbers, operators, input, HashSet, REPL, classes, objects, traits, inheritance
Module 4	Scala exceptions; Apache Spark (RDD, DataFrame, SQL, MLlib, streaming, GraphX)
Module 5	Language selection criteria as a weighted scoring matrix
Size	32 tables, 431 rows (24 Python tables, 8 Scala tables)

Watch out. Every row was run, except the few marked 'not run' in the tables and listed on the last page. The network examples were checked against local stand-ins, not the live sites.

Contents

How the syllabus was read 4

Module 1: Python Data Structures (8 hours) . . . 4

Module 2: Python Libraries (9 hours) . . . 4

Module 3: Scala Data Structures and Object-Oriented Programming (12 hours) . . . 5

Module 4: Scala Libraries and Spark Basics (8 hours) . . . 6

Module 5: Programming language selection criteria (6 hours) . . . 6

Module 6: Contemporary Issues (2 hours) . . . 6

The 20 indicative experiments and the cheat-sheet table that solves each . . . 6

Module 1: Python Data Structures (8 hours) 8

1.1 Condition and branching statements, loops . . . 8

1.2 Built-in data structures: list, tuple, dictionary, set . . . 8

1.3 User-defined data structures: Stack, Queue, Priority Queue . . . 9

1.4 String handling methods . . . 11

1.5 Exception handling in Python . . . 11

1.6 Object-oriented concepts in Python . . . 12

1.7 File handling: read and write . . . 14

1.8 APIs and data collection: simple API and REST APIs (HTTP requests) . . . 15

1.9 NLTK: text processing . . . 16

1.10 ChatterBot: a rule-free retrieval chatbot . . . 16

Module 2: Python Libraries (9 hours) 18

2.1 pandas Series . . . 18

2.2 DataFrame: built-in functions, data operations, filtering, extraction . . . 18

2.3 Working with text data in pandas . . . 19

2.4 Merging DataFrames: merge, join, concat, pivot . . . 20

2.5 SciPy: statistics, optimisation, integration, interpolation, linear algebra . . . 21

2.6 Scrapy: a spider with items, a pipeline and pagination . . . 21

2.7 scikit-learn: the supervised learning workflow . . . 22

2.8 XGBoost: gradient-boosted trees . . . 23

2.9 Keras: a neural network with the Sequential API . . . 24

2.10 TensorFlow: tensors, gradients, tf.function, tf.data . . . 24

2.11 PyTorch: tensors, autograd, nn.Module, training loop . . . 25

2.12 Folium: interactive maps . . . 26

2.13 More Matplotlib and seaborn plots . . . 27

Module 3: Scala Data Structures and Object-Oriented Programming (12 hours) 28

3.1 Functions: the expanded function format . . . 28

3.2 Variables, strings, numbers and types . . . 29

3.3 Operators, booleans, control flow and user input . . . 30

3.4 Arrays, HashSet and the other mutable collections; the REPL . . . 31

3.5 Classes and objects: constructors, overloading, this, singleton, companion, case classes, access modifiers . . . 33

3.6 Inheritance, overriding, final, abstract classes, traits and mixins . . . 35

Module 4: Scala Libraries and Spark Basics (8 hours) 37

4.1 Exception handling in Scala . . . 37

4.2 Apache Spark: architecture, RDDs, DataFrames, SQL, MLlib, streaming, GraphX . . . 38

Module 5: Programming language selection criteria (6 hours) 40

5.1 Language selection: a weighted decision matrix . . . 40

What was verified and what was not 41

Verified by running . . . 41

Not verified, or only partly . . . 41

How the syllabus was read

The topics below are the phrases of the syllabus PDF (*MACSE502_PROGRAMMING-FOR-DATA-SCIENCE_ETH_1.0_83_MACSE502.pdf*), module by module. A script checked that all 86 topic phrases and all 20 experiment titles occur in the text of that PDF (none missing). The lab-based cheat sheet covers what the 20 indicative experiments ask for. Everything else that the syllabus names has code in this file.

Module 1: Python Data Structures (8 hours)

Syllabus topic	Status	Where the code is
Condition and Branching Statements	**new here**	This file, 1.1
List, Tuple, Dictionary, Set	**new here**	This file, 1.2
Stack, Queue, Priority Queue	**new here**	This file, 1.3
String handling methods	**new here**	This file, 1.4
Exception Handling	**new here**	This file, 1.5
Object-Oriented Concepts	**new here**	This file, 1.6
APIs and Data Collection	**new here**	This file, 1.8
Simple API and REST APIs	**new here**	This file, 1.8
File Handling	**new here**	This file, 1.7
NLTK	**new here**	This file, 1.9
ChatterBot	**new here**	This file, 1.10

Module 2: Python Libraries (9 hours)

Syllabus topic	Status	Where the code is
Series	**new here**	This file, 2.1
DataFrame	**basics in the cheat sheet, more here**	Cheat sheet table A; this file, 2.2
Handling Missing Values	**basics in the cheat sheet, more here**	Cheat sheet table B; this file, 2.1
Built-in functions	**new here**	This file, 2.2
Data Operations	**new here**	This file, 2.2
Filtering Data in DataFrame	**basics in the cheat sheet, more here**	Cheat sheet table A; this file, 2.2
Data Extraction	**new here**	This file, 2.2
Working with Text Data	**new here**	This file, 2.3
Merging DataFrames	**new here**	This file, 2.4
Scrapy	**new here**	This file, 2.6
Beautiful Soup	**already in the cheat sheet**	Cheat sheet table E
NumPy	**already in the cheat sheet**	Cheat sheet table C
SciPy	**new here**	This file, 2.5
Pandas	**already in the cheat sheet**	Cheat sheet table A, B
Keras	**new here**	This file, 2.9
Scikit - Learn	**basics in the cheat sheet, more here**	Cheat sheet table H (PCA, K-means); this file, 2.7
PyTorch	**new here**	This file, 2.11
TensorFlow	**new here**	This file, 2.10
XGBoost	**new here**	This file, 2.8
Matplotlib	**basics in the cheat sheet, more here**	Cheat sheet table D; this file, 2.13
Seaborn	**basics in the cheat sheet, more here**	Cheat sheet table D; this file, 2.13

Syllabus topic	Status	Where the code is
Bokeh	**already in the cheat sheet**	Cheat sheet table G
Plotly	**already in the cheat sheet**	Cheat sheet table F
Folium	**new here**	This file, 2.12

Module 3: Scala Data Structures and Object-Oriented Programming (12 hours)

Syllabus topic	Status	Where the code is
Expanded Function Format	**new here**	This file, 3.1
Variables and Strings	**new here**	This file, 3.2
Getting user input	**new here**	This file, 3.3
Numbers	**new here**	This file, 3.2
Variable types	**new here**	This file, 3.2
Operators	**new here**	This file, 3.3
Booleans	**new here**	This file, 3.3
Arrays	**basics in the cheat sheet, more here**	Cheat sheet table S1; this file, 3.4
Lists	**already in the cheat sheet**	Cheat sheet table S1
Tuple	**already in the cheat sheet**	Cheat sheet table S1
Sets	**already in the cheat sheet**	Cheat sheet table S1
Hash set	**new here**	This file, 3.4
Maps	**already in the cheat sheet**	Cheat sheet table S1
Functional Combinators	**already in the cheat sheet**	Cheat sheet table S2
Scala Object and Class	**new here**	This file, 3.5
Anonymous object	**new here**	This file, 3.5
Singleton	**new here**	This file, 3.5
Companion Object	**new here**	This file, 3.5
Case Classes	**basics in the cheat sheet, more here**	Cheat sheet table S2; this file, 3.5
Constructors	**new here**	This file, 3.5
Method Overloading	**new here**	This file, 3.5
This Keyword	**new here**	This file, 3.5
Inheritance	**new here**	This file, 3.6
Field Overriding	**new here**	This file, 3.6
Final	**new here**	This file, 3.6
Abstract Class	**new here**	This file, 3.6
Trait	**new here**	This file, 3.6
Trait Mixins	**new here**	This file, 3.6
Access Modifiers	**new here**	This file, 3.5
Scala Array-REPL	**new here**	This file, 3.4

Module 4: Scala Libraries and Spark Basics (8 hours)

Syllabus topic	Status	Where the code is
Breeze	**already in the cheat sheet**	Cheat sheet table S4
saddle	**already in the cheat sheet**	Cheat sheet table S3
Exception Handling	**new here**	This file, 4.1
Apache Spark Architecture	**new here**	This file, 4.2
Spark Big Data	**new here**	This file, 4.2
Apache Spark Components	**new here**	This file, 4.2

Module 5: Programming language selection criteria (6 hours)

Syllabus topic	Status	Where the code is
Size of the Deployment	**new here**	This file, 5.1 (as a scoring matrix)
Security	**new here**	This file, 5.1 (as a scoring matrix)
Skill Set	**new here**	This file, 5.1 (as a scoring matrix)
targeted platform	**new here**	This file, 5.1 (as a scoring matrix)
elasticity of a language	**new here**	This file, 5.1 (as a scoring matrix)
time to production	**new here**	This file, 5.1 (as a scoring matrix)
performance	**new here**	This file, 5.1 (as a scoring matrix)
support and community	**new here**	This file, 5.1 (as a scoring matrix)
Purpose	**new here**	This file, 5.1 (as a scoring matrix)
Programmer experience	**new here**	This file, 5.1 (as a scoring matrix)
Ease of Development and Maintenance	**new here**	This file, 5.1 (as a scoring matrix)
Efficiency	**new here**	This file, 5.1 (as a scoring matrix)
Availability of an IDE	**new here**	This file, 5.1 (as a scoring matrix)
Error Checking and Diagnosis	**new here**	This file, 5.1 (as a scoring matrix)

Module 6: Contemporary Issues (2 hours)

Syllabus topic	Status	Where the code is
Contemporary Issues	**no code**	the syllabus lists no topics

The 20 indicative experiments and the cheat-sheet table that solves each

No	Experiment	Cheat sheet table
1	Basic Data Manipulation with Pandas	A (pandas basics)
2	Data Cleaning and Preprocessing with Pandas	B (cleaning and missing values)
3	Creating and Manipulating Arrays	C (NumPy)
4	Feature Engineering, Exploratory Data Analysis	D (Matplotlib / seaborn EDA)
5	Scrape data from a webpage	E (Beautiful Soup)
6	Interactive Bar Chart with Plotly	F (Plotly)
7	Interactive Scatter Plot with Plotly	F (Plotly)
8	Interactive Line Plot with Bokeh	G (Bokeh)
9	Interactive Bar Chart with Bokeh	G (Bokeh)
10	Interactive Scatter Plot with Bokeh	G (Bokeh)
11	Visualizing Clusters with scikit-learn and Plotly	H (PCA / K-means)
12	Visualizing PCA with scikit-learn and Plotly	H (PCA / K-means)
13	Introduction to arrays in Scala	S1 (Scala collections)
14	Understanding lists, sets, and tuples in Scala	S1 (Scala collections)
15	Handling collisions and resolving conflicts in HashMap	S1 (Scala collections)

No	Experiment	Cheat sheet table
16	Exploring advanced functional combinators	S2 (combinators and case class)
17	Case Classes	S2 (combinators and case class)
18	Use Saddle to load and manipulate a dataset	S3 (Saddle)
19	Use Breeze library to calculate total revenue	S4 (Breeze and Plotly.Scala)
20	Analyse fruit prices using Spire	S5 (Spire)

Module 1: Python Data Structures (8 hours)

1.1 Condition and branching statements, loops

Syllabus: Module 1: Condition and Branching Statements. 12 rows, in running order. Install: standard libraries only.

T.No	Task Desc	Program code
1	Inputs used below	`marks = 72` `attendance = 81`

2	if / elif / else	`if marks >= 90:`
		`  grade = 'S'`
		`elif marks >= 80:`
		`  grade = 'A'`
		`elif marks >= 70:`
		`  grade = 'B'`
		`else:`
		`  grade = 'C'`
		`print(grade)`

3	Combine conditions with and / or / not	`if marks >= 40 and attendance >= 75:`
		`  result = 'Pass'`
		`elif marks >= 40:`
		`  result = 'Detained (attendance)'`
		`else:`
		`  result = 'Fail'`
		`print(result)`

4	Conditional expression (one-line if)	`status = 'Eligible' if attendance >= 75 else 'Not eligible'` `print(status)`
5	Chained comparison, membership, identity	`print(70 <= marks < 80, 'B' in ['A', 'B'], marks is None, not marks > 100)`
6	Truthiness: what counts as False	`print(bool([]), bool(''), bool(0), bool(None), bool([0]), bool('False'))`

7	for loop with range and enumerate	`for i in range(1, 6, 2):`
		`  print(i, end=' ')`
		`for position, name in enumerate(['Asha', 'Ravi', 'Meena'], start=1):`
		`  print(position, name)`

8	while loop with break and continue	`n, total = 0, 0`
		`while True:`
		`  n += 1`
		`  if n % 2 == 0:`
		`  continue`
		`  if n > 9:`
		`  break`
		`  total += n`
		`print(total)`

9	for ... else (else runs when there was no break)	`for n in [4, 6, 8]:`
		`  if n % 2:`
		`  print('odd number found')`
		`  break`
		`else:`
		`  print('no odd number')`

10	Nested loops: multiplication table	`for i in range(1, 4):` `  print([i * j for j in range(1, 6)])`
11	List comprehension with a condition	`even_squares = [x * x for x in range(10) if x % 2 == 0]` `print(even_squares)`

12	match / case (Python 3.10+)	`command = 'stop'`
		`match command:`
		`  case 'go':`
		`  print('moving')`
		`  case 'stop' | 'halt':`
		`  print('stopping')`
		`  case _:`
		`  print('unknown')`

1.2 Built-in data structures: list, tuple, dictionary, set

Syllabus: Module 1: List, Tuple, Dictionary, Set. 13 rows, in running order. Install: standard libraries only.

T.No	Task Desc	Program code
1	List: create, index, slice	`nums = [5, 3, 8, 1]` `print(nums[0], nums[-1], nums[1:3], nums[::-1])`

2	List methods	`nums.append(9)`
		`nums.insert(1, 7)`
		`nums.extend([2, 2])`
		`nums.remove(2)`
		`last = nums.pop()`
		`nums.sort(reverse=True)`
		`print(nums, len(nums), nums.index(8), nums.count(2))`

T.No	Task Desc	Program code

3	sorted(), reversed(), min/max/sum, copy	`ordered = sorted(nums)`
		`print(ordered, min(nums), max(nums), sum(nums), list(reversed(nums)))`
		`backup = nums.copy()`

4	Tuple: packing, unpacking, immutability	`point = (3, 4)`
		`x, y = point`
		`first, *rest = (1, 2, 3, 4)`
		`single = (5,)`
		`print(x, y, first, rest, type(single).__name__)`
		`try:`
		`  point[0] = 9`
		`except TypeError as e:`
		`  print('error:', e)`

5	Dictionary: create, read, add, update, delete	`student = {'name': 'Asha', 'age': 20}`
		`print(student['name'], student.get('city', 'unknown'))`
		`student['city'] = 'Pune'`
		`student.update({'age': 21})`
		`del student['age']`
		`print(student, list(student.keys()), list(student.values()))`

6	Iterate over a dictionary	`for key, value in student.items():` `  print(key, '->', value)`

7	Count words with a dictionary	`words = ['a', 'b', 'a', 'c', 'a']`
		`counts = {}`
		`for w in words:`
		`  counts[w] = counts.get(w, 0) + 1`
		`print(counts)`

8	Dictionary comprehension	`lengths = {w: len(w) for w in ['data', 'science']}` `print(lengths)`

9	Counter and defaultdict	`from collections import Counter, defaultdict`
		`print(Counter(words).most_common(1))`
		`groups = defaultdict(list)`
		`for w in words:`
		`  groups[w].append(w)`
		`print(dict(groups))`

10	Set: create, add, remove, operations	`a = {1, 2, 3, 3}`
		`b = {3, 4}`
		`print(a | b, a & b, a - b, a ^ b, {1, 2} <= a)`
		`a.add(10)`
		`a.discard(99)`

11	Remove duplicates with a set	`unique = sorted(set([3, 1, 3, 2, 1]))` `print(unique)`

12	Nested structures: list of dictionaries	`records = [{'name': 'Asha', 'marks': [80, 90]}, {'name': 'Ravi', 'marks': [70, 60]}]`
		`average = {r['name']: sum(r['marks']) / len(r['marks']) for r in records}`
		`print(average)`

13	Shallow copy versus deep copy	`import copy`
		`a = [1, [2, 3]]`
		`shallow = a.copy()`
		`deep = copy.deepcopy(a)`
		`a[1].append(4)`
		`print(shallow, deep)`

Note. A list is ordered and mutable, a tuple is ordered and immutable, a dict maps keys to values, a set is unordered with unique items. Keys of a dict and items of a set must be hashable (no lists).

1.3 User-defined data structures: Stack, Queue, Priority Queue

Syllabus: Module 1: Stack, Queue, Priority Queue. 9 rows, in running order. Install: standard libraries only.

T.No	Task Desc	Program code

1	Stack class (last in, first out)	`class Stack:`
		`  def __init__(self):`
		`  self._items = []`
		
		`  def push(self, x):`
		`  self._items.append(x)`
		
		`  def pop(self):`
		`  if self.is_empty():`
		`  raise IndexError('pop from empty stack')`
		`  return self._items.pop()`
		
		`  def peek(self):`
		`  return self._items[-1]`
		
		`  def is_empty(self):`
		`  return not self._items`
		
		`  def __len__(self):`
		`  return len(self._items)`

T.No	Task Desc	Program code

2	Use the stack: balanced brackets	`def balanced(text):`
		`  pairs = {')': '(', ']': '[', '}': '{'}`
		`  s = Stack()`
		`  for ch in text:`
		`  if ch in '([{':`
		`  s.push(ch)`
		`  elif ch in pairs:`
		`  if s.is_empty() or s.pop() != pairs[ch]:`
		`  return False`
		`  return s.is_empty()`
		
		`print(balanced('{[()]}'), balanced('(]'))`

3	Queue class (first in, first out) on a deque	`from collections import deque`
		
		`class Queue:`
		`  def __init__(self):`
		`  self._items = deque()`
		
		`  def enqueue(self, x):`
		`  self._items.append(x)`
		
		`  def dequeue(self):`
		`  return self._items.popleft()`
		
		`  def is_empty(self):`
		`  return not self._items`
		
		`  def __len__(self):`
		`  return len(self._items)`

4	Use the queue	`q = Queue()`
		`for name in ['Asha', 'Ravi', 'Meena']:`
		`  q.enqueue(name)`
		`print(q.dequeue(), len(q))`

5	Priority queue with heapq (smallest first)	`import heapq`
		`pq = []`
		`heapq.heappush(pq, (2, 'write report'))`
		`heapq.heappush(pq, (1, 'fix bug'))`
		`heapq.heappush(pq, (3, 'lunch'))`
		`while pq:`
		`  priority, task = heapq.heappop(pq)`
		`  print(priority, task)`

6	PriorityQueue class (ties keep arrival order)	`import itertools`
		
		`class PriorityQueue:`
		`  def __init__(self):`
		`  self._heap = []`
		`  self._count = itertools.count()`
		
		`  def push(self, item, priority):`
		`  heapq.heappush(self._heap, (priority, next(self._count), item))`
		
		`  def pop(self):`
		`  return heapq.heappop(self._heap)[2]`
		
		`  def __len__(self):`
		`  return len(self._heap)`

7	Use it	`pq2 = PriorityQueue()`
		`pq2.push('B', 2)`
		`pq2.push('A', 1)`
		`pq2.push('C', 2)`
		`print([pq2.pop() for _ in range(3)])`

8	Max-heap trick: store negative priorities	`maxheap = []`
		`for v in [4, 9, 2]:`
		`  heapq.heappush(maxheap, -v)`
		`print(-heapq.heappop(maxheap))`

9	Ready-made versions in the standard library	`from queue import Queue as FifoQueue, LifoQueue, PriorityQueue as PQ`
		`fifo, lifo, pq3 = FifoQueue(), LifoQueue(), PQ()`
		`for x in (1, 2, 3):`
		`  fifo.put(x)`
		`  lifo.put(x)`
		`pq3.put((2, 'b'))`
		`pq3.put((1, 'a'))`
		`print(fifo.get(), lifo.get(), pq3.get())`

Note. list.pop(0) is slow (O(n)); a deque gives O(1) at both ends. heapq is a min-heap: push and pop are O(log n).

1.4 String handling methods

Syllabus: Module 1: String handling methods. 12 rows, in running order. Install: standard libraries only.

T.No	Task Desc	Program code
1	Length and trimming	`s = '  Data Science with Python  '` `print(len(s), repr(s.strip()), repr(s.lstrip()), repr(s.rstrip()))`
2	Change case	`t = s.strip()` `print(t.upper(), t.lower(), t.title(), t.swapcase(), t.capitalize())`
3	Search	`print(t.find('Science'), t.find('xyz'), t.index('with'), t.count('a'),` `  t.startswith('Data'), t.endswith('on'))`

4	split, join, replace	`words = t.split()`
		`print(words)`
		`print('-'.join(words))`
		`print(t.replace('Python', 'Scala'))`

5	Slicing and reversing	`print(t[0:4], t[-6:], t[::-1], t[::2])`
6	Character tests	`print('123'.isdigit(), 'abc'.isalpha(), 'ab12'.isalnum(), ' '.isspace(),` `  'Abc'.istitle())`

7	Formatting: f-string, format(), %	`name, score = 'Asha', 92.456`
		`print(f'{name} scored {score:.1f}')`
		`print('{} scored {:.2f}'.format(name, score))`
		`print('%s scored %d' % (name, score))`
		`print(f'{score:>10.2f}|{name:<8}|{42:05d}')`

8	Parse a CSV-like line	`line = 'Asha,20,Pune'`
		`name, age, city = line.split(',')`
		`print(line.partition(','), line.rsplit(',', 1))`

9	Strings are immutable	`try:`
		`  t[0] = 'd'`
		`except TypeError as e:`
		`  print('error:', e)`
		`print('apple' < 'banana', 'a' * 3, 'Sci' in t)`

10	Palindrome check	`def is_palindrome(text):`
		`  clean = ''.join(ch.lower() for ch in text if ch.isalnum())`
		`  return clean == clean[::-1]`
		
		`print(is_palindrome('A man, a plan, a canal: Panama'))`

11	Regular expressions with re	`import re`
		`print(re.findall(r'\d+', 'Order 66 shipped on 12 May'))`
		`print(re.sub(r'\s+', ' ', 'too   many    spaces'))`
		`m = re.search(r'(\w+)@(\w+)\.com', 'mail asha@vit.com now')`
		`print(m.group(1), m.group(2))`

12	Escapes, raw strings, multi-line text	`text = 'line1\nline2\tTabbed'`
		`print(text.splitlines())`
		`print(r'C:\new\table')`
		`poem = """first`
		`second"""`
		`print(len(poem.splitlines()))`

1.5 Exception handling in Python

Syllabus: Module 1: Exception Handling. 12 rows, in running order. Install: standard libraries only.

T.No	Task Desc	Program code

1	try / except	`try:`
		`  value = int('abc')`
		`except ValueError:`
		`  value = 0`
		`print(value)`

2	Several except blocks	`def divide(a, b):`
		`  try:`
		`  return a / b`
		`  except ZeroDivisionError:`
		`  return float('inf')`
		`  except TypeError as e:`
		`  return f'bad types: {e}'`
		
		`print(divide(1, 0), divide('a', 2))`

3	else and finally	`def read_number(text):`
		`  try:`
		`  n = int(text)`
		`  except ValueError:`
		`  print('not a number')`
		`  else:`
		`  print('parsed', n)`
		`  return n`
		`  finally:`
		`  print('done')`
		
		`read_number('42')`
		`read_number('x')`

T.No	Task Desc	Program code

4	raise an exception yourself	`def set_age(age):`
		`  if age < 0:`
		`  raise ValueError('age cannot be negative')`
		`  return age`
		
		`try:`
		`  set_age(-1)`
		`except ValueError as e:`
		`  print('caught:', e)`

5	Custom exception class	`class InsufficientFunds(Exception):`
		`  def __init__(self, balance, amount):`
		`  super().__init__(f'balance {balance} is less than {amount}')`
		`  self.balance, self.amount = balance, amount`

6	Use the custom exception	`def withdraw(balance, amount):`
		`  if amount > balance:`
		`  raise InsufficientFunds(balance, amount)`
		`  return balance - amount`
		
		`try:`
		`  withdraw(100, 250)`
		`except InsufficientFunds as e:`
		`  print(e, '| short by', e.amount - e.balance)`

7	Catch several types at once	`try:`
		`  [1, 2][5]`
		`except (KeyError, IndexError) as e:`
		`  print(type(e).__name__, e)`

8	Exception chaining with raise ... from	`try:`
		`  try:`
		`  {}['x']`
		`  except KeyError as e:`
		`  raise RuntimeError('config value missing') from e`
		`except RuntimeError as err:`
		`  print(err, '| caused by', repr(err.__cause__))`

9	assert	`try:`
		`  assert 1 + 1 == 3, 'arithmetic is broken'`
		`except AssertionError as e:`
		`  print('assertion failed:', e)`

10	Exception hierarchy	`print([k.__name__ for k in ZeroDivisionError.__mro__])`

11	File errors and the with statement	`try:`
		`  with open('missing.txt') as f:`
		`  data = f.read()`
		`except FileNotFoundError as e:`
		`  print('no such file:', e.filename)`

12	Traceback text for logging	`import traceback`
		`try:`
		`  1 / 0`
		`except ZeroDivisionError:`
		`  print(traceback.format_exc().splitlines()[-1])`

Note. Order the except blocks from the most specific to the most general. A bare `except:` also hides KeyboardInterrupt, so catch Exception at most.

1.6 Object-oriented concepts in Python

Syllabus: Module 1: Object-Oriented Concepts. 11 rows, in running order. Install: standard libraries only.

T.No	Task Desc	Program code

1	Class with class and instance attributes	`class Student:`
		`  college = 'VIT'  # class attribute: shared by all objects`
		
		`  def __init__(self, name, marks):`
		`  self.name = name  # instance attributes`
		`  self.marks = marks`
		
		`  def average(self):`
		`  return sum(self.marks) / len(self.marks)`
		
		`  def __str__(self):`
		`  return f'{self.name} ({self.average():.1f})'`

2	Create and use objects	`s1 = Student('Asha', [80, 90])` `print(s1, s1.college, s1.average())`

T.No	Task Desc	Program code

3	Inheritance, super() and overriding	`class PGStudent(Student):`
		`  def __init__(self, name, marks, thesis):`
		`  super().__init__(name, marks)`
		`  self.thesis = thesis`
		
		`  def __str__(self):`
		`  return super().__str__() + ' | thesis: ' + self.thesis`
		
		`print(PGStudent('Ravi', [70, 75], 'NLP'))`

4	Abstract class and polymorphism	`from abc import ABC, abstractmethod`
		`import math`
		
		`class Shape(ABC):`
		`  @abstractmethod`
		`  def area(self):`
		`  ...`
		
		`class Circle(Shape):`
		`  def __init__(self, r):`
		`  self.r = r`
		
		`  def area(self):`
		`  return math.pi * self.r ** 2`
		
		`class Rect(Shape):`
		`  def __init__(self, w, h):`
		`  self.w, self.h = w, h`
		
		`  def area(self):`
		`  return self.w * self.h`
		
		`for shape in [Circle(1), Rect(2, 3)]:`
		`  print(type(shape).__name__, round(shape.area(), 2))`

5	The abstract class cannot be instantiated	`try:`
		`  Shape()`
		`except TypeError as e:`
		`  print('error:', e)`

6	Encapsulation: private attribute and property	`class Account:`
		`  def __init__(self, balance):`
		`  self.__balance = balance  # name-mangled to _Account__balance`
		
		`  @property`
		`  def balance(self):`
		`  return self.__balance`
		
		`  @balance.setter`
		`  def balance(self, value):`
		`  if value < 0:`
		`  raise ValueError('negative balance')`
		`  self.__balance = value`
		
		`acc = Account(100)`
		`acc.balance = 150`
		`print(acc.balance)`

7	classmethod and staticmethod	`class Temp:`
		`  count = 0`
		
		`  def __init__(self, celsius):`
		`  self.celsius = celsius`
		`  Temp.count += 1`
		
		`  @classmethod`
		`  def from_fahrenheit(cls, f):`
		`  return cls((f - 32) * 5 / 9)`
		
		`  @staticmethod`
		`  def is_valid(c):`
		`  return c >= -273.15`
		
		`t = Temp.from_fahrenheit(212)`
		`print(t.celsius, Temp.is_valid(-300), Temp.count)`

8	Special (dunder) methods	`class Vector:`
		`  def __init__(self, x, y):`
		`  self.x, self.y = x, y`
		
		`  def __add__(self, other):`
		`  return Vector(self.x + other.x, self.y + other.y)`
		
		`  def __eq__(self, other):`
		`  return (self.x, self.y) == (other.x, other.y)`
		
		`  def __repr__(self):`
		`  return f'Vector({self.x}, {self.y})'`
		
		`print(Vector(1, 2) + Vector(3, 4), Vector(1, 2) == Vector(1, 2))`

T.No	Task Desc	Program code

9	Multiple inheritance and the method resolution order	`class A:`
		`  def who(self): return 'A'`
		`class B(A):`
		`  def who(self): return 'B'`
		`class C(A):`
		`  def who(self): return 'C'`
		`class D(B, C):`
		`  pass`
		
		`print(D().who(), [k.__name__ for k in D.__mro__])`

10	isinstance, issubclass, hasattr, getattr	`print(isinstance(s1, Student), issubclass(PGStudent, Student), hasattr(s1, 'name'), getattr(s1, 'city', 'n/a'))`

11	dataclass: less boilerplate	`from dataclasses import dataclass`
		
		`@dataclass`
		`class Point:`
		`  x: float`
		`  y: float = 0.0`
		
		`print(Point(1.5), Point(1, 2) == Point(1, 2))`

Note. Python has no real private members: a leading underscore is a convention, a double underscore only mangles the name.

1.7 File handling: read and write

Syllabus: Module 1: File Handling - Read/Write. 12 rows, in running order. Install: standard libraries only.

T.No	Task Desc	Program code

1	Write a text file	`with open('notes.txt', 'w', encoding='utf-8') as f:`
		`  f.write('line one\n')`
		`  f.write('line two\n')`
		`  f.writelines(['line three\n', 'line four\n'])`

2	Read the whole file	`with open('notes.txt', 'r', encoding='utf-8') as f:`
		`  content = f.read()`
		`print(len(content), content.count('\n'))`

3	Read line by line	`with open('notes.txt') as f:`
		`  for number, line in enumerate(f, start=1):`
		`  print(number, line.rstrip())`

4	Append to the file	`with open('notes.txt', 'a') as f:` `  f.write('line five\n')`

5	readline, readlines, seek	`with open('notes.txt') as f:`
		`  first = f.readline()`
		`  f.seek(0)`
		`  all_lines = f.readlines()`
		`print(first.strip(), len(all_lines))`

6	Paths and existence checks	`import os`
		`from pathlib import Path`
		`print(os.path.exists('notes.txt'), Path('notes.txt').suffix,`
		`  Path('notes.txt').stat().st_size > 0)`
		`print(Path('notes.txt').read_text().count('line'))`

7	CSV: write and read with the csv module	`import csv`
		`with open('marks.csv', 'w', newline='') as f:`
		`  writer = csv.writer(f)`
		`  writer.writerow(['name', 'marks'])`
		`  writer.writerows([['Asha', 90], ['Ravi', 75]])`
		`with open('marks.csv', newline='') as f:`
		`  for row in csv.DictReader(f):`
		`  print(row['name'], int(row['marks']) + 5)`

8	JSON: dump and load	`import json`
		`data = {'name': 'Asha', 'courses': ['DS', 'ML'], 'active': True}`
		`with open('student.json', 'w') as f:`
		`  json.dump(data, f, indent=2)`
		`with open('student.json') as f:`
		`  loaded = json.load(f)`
		`print(loaded == data, json.dumps(data))`

9	pandas: read_csv / to_csv / to_json	`import pandas as pd`
		`df = pd.read_csv('marks.csv')`
		`df['grade'] = ['A', 'B']`
		`df.to_csv('marks_out.csv', index=False)`
		`df.to_json('marks.json', orient='records')`
		`print(pd.read_json('marks.json').shape)`

T.No	Task Desc	Program code

10	Binary files and pickle	`import pickle`
		`with open('data.bin', 'wb') as f:`
		`  f.write(bytes([72, 105]))`
		`with open('data.bin', 'rb') as f:`
		`  print(f.read())`
		`with open('obj.pkl', 'wb') as f:`
		`  pickle.dump({'a': [1, 2]}, f)`
		`with open('obj.pkl', 'rb') as f:`
		`  print(pickle.load(f))`

11	List and delete files	`print(sorted(name for name in os.listdir('.') if name.endswith('.csv')))` `os.remove('data.bin')`

12	Word frequency of a text file	`from collections import Counter`
		`with open('notes.txt') as f:`
		`  freq = Counter(f.read().split())`
		`print(freq.most_common(2))`

Note. Modes: 'r' read (default), 'w' write (erases the file), 'a' append, 'x' create only, add 'b' for binary. Always use `with` so the file is closed even after an error.

1.8 APIs and data collection: simple API and REST APIs (HTTP requests)

Syllabus: Module 1: APIs and Data Collection, Simple API and REST APIs - HTTP Requests. 13 rows, in running order. Install: pip install requests pandas.

T.No	Task Desc	Program code
1	Load packages	`import requests` `import pandas as pd`
2	Base URL of a public test API	`BASE = 'https://jsonplaceholder.typicode.com'`
3	GET request: status code, headers	`r = requests.get(BASE + '/posts/1', timeout=10)` `print(r.status_code, r.ok, r.headers['Content-Type'])`
4	Parse the JSON body	`post = r.json()` `print(post['id'], post['title'])`

5	Query parameters	`r = requests.get(BASE + '/posts', params={'userId': 1}, timeout=10)`
		`print(r.url)`
		`posts = r.json()`
		`print(type(posts).__name__, len(posts) > 0)`

6	Send headers (content type, token)	`headers = {'Accept': 'application/json', 'Authorization': 'Bearer <token>'}`
		`r = requests.get(BASE + '/posts/1', headers=headers, timeout=10)`
		`print(r.status_code)`

7	POST: create a resource with a JSON body	`new_post = {'title': 'hello', 'body': 'first post', 'userId': 1}`
		`r = requests.post(BASE + '/posts', json=new_post, timeout=10)`
		`print(r.status_code, r.json()['id'])`

8	PUT and DELETE	`r = requests.put(BASE + '/posts/1', json={'id': 1, 'title': 'edited', 'body':`
		`  'x', 'userId': 1}, timeout=10)`
		`print(r.status_code)`
		`print(requests.delete(BASE + '/posts/1', timeout=10).status_code)`

9	Error status codes and raise_for_status()	`r = requests.get(BASE + '/posts/99999', timeout=10)`
		`print(r.status_code)`
		`try:`
		`  r.raise_for_status()`
		`except requests.exceptions.HTTPError as e:`
		`  print('HTTP error:', e)`

10	Network errors	`try:`
		`  requests.get('http://127.0.0.1:9', timeout=2)`
		`except requests.exceptions.RequestException as e:`
		`  print('request failed:', type(e).__name__)`

11	Reuse a connection with a Session	`s = requests.Session()`
		`s.headers.update({'Accept': 'application/json'})`
		`print(s.get(BASE + '/posts/1', timeout=10).status_code)`

12	JSON to DataFrame to CSV	`df = pd.DataFrame(requests.get(BASE + '/posts', timeout=10).json())`
		`print(df.columns.tolist())`
		`df.to_csv('posts.csv', index=False)`

13	The same GET with only the standard library	`import json`
		`import urllib.request`
		`with urllib.request.urlopen(BASE + '/posts/1') as resp:`
		`  print(resp.status, json.load(resp)['userId'])`

Note. GET reads, POST creates, PUT replaces, PATCH changes part, DELETE removes. 2xx = success, 3xx = redirect, 4xx = client error (404 not found, 401 unauthorised), 5xx = server error. Always pass timeout=. Verified against a local stub server that copies the routes of jsonplaceholder.typicode.com (the live site was not contacted).

1.9 NLTK: text processing

Syllabus: Module 1: Frameworks and Libraries - NLTK. 11 rows, in running order. Install: pip install nltk scikit-learn.

T.No	Task Desc	Program code

1	Download the resources (once) *(not run: needs internet (the data was already installed))*	`import nltk`
		`for pkg in ['punkt', 'punkt_tab', 'stopwords', 'wordnet',`
		`    'averaged_perceptron_tagger_eng']:`
		`    nltk.download(pkg, quiet=True)`

2	Split into sentences and words	`from nltk.tokenize import sent_tokenize, word_tokenize`
		`text = "Data science is fun. Python makes it easier, doesn't it?"`
		`sentences = sent_tokenize(text)`
		`tokens = word_tokenize(text)`
		`print(sentences)`
		`print(tokens)`

3	Lower-case, keep words, remove stopwords	`from nltk.corpus import stopwords`
		`stop = set(stopwords.words('english'))`
		`words = [t.lower() for t in tokens if t.isalpha() and t.lower() not in stop]`
		`print(words)`

4	Stemming (cuts suffixes)	`from nltk.stem import PorterStemmer`
		`ps = PorterStemmer()`
		`print([ps.stem(w) for w in ['running', 'flies', 'studies', 'easier']])`

5	Lemmatization (dictionary form, needs the part of speech)	`from nltk.stem import WordNetLemmatizer`
		`wl = WordNetLemmatizer()`
		`print(wl.lemmatize('studies'), wl.lemmatize('running', pos='v'),`
		`    wl.lemmatize('better', pos='a'))`

6	Part-of-speech tagging	`from nltk import pos_tag` `print(pos_tag(word_tokenize('The quick fox jumps')))`

7	Word frequencies	`from nltk import FreqDist`
		`fd = FreqDist(word_tokenize('data data science data python science'))`
		`print(fd.most_common(2), fd['data'])`

8	n-grams	`from nltk import bigrams, ngrams` `print(list(bigrams(['a', 'b', 'c'])), list(ngrams(['a', 'b', 'c', 'd'], 3)))`
9	Tokenize with a regular expression	`from nltk.tokenize import RegexpTokenizer` `print(RegexpTokenizer(r'\w+').tokenize('hello, world! 2026'))`

10	WordNet: meanings	`from nltk.corpus import wordnet`
		`syn = wordnet.synsets('good')[0]`
		`print(syn.name(), '|', syn.definition())`

11	Bag of words with scikit-learn	`from sklearn.feature_extraction.text import CountVectorizer`
		`cv = CountVectorizer()`
		`X = cv.fit_transform(['data science is fun', 'science needs data'])`
		`print(cv.get_feature_names_out().tolist())`
		`print(X.toarray())`

Note. Typical pipeline: tokenize, lower-case, remove stopwords and punctuation, stem or lemmatize, then count (bag of words / TF-IDF).

1.10 ChatterBot: a rule-free retrieval chatbot

Syllabus: Module 1: Frameworks and Libraries - ChatterBot. 7 rows, in running order. Needs ChatterBot (its own environment). Install: pip install chatterbot spacy, then python -m spacy download en_core_web_sm.

T.No	Task Desc	Program code

1	Load packages	`import logging`
		`from chatterbot import ChatBot`
		`from chatterbot.trainers import ListTrainer`
		`logging.disable(logging.CRITICAL)`

2	Create the bot (SQLite storage, fallback answer)	`bot = ChatBot('CampusBot',`
		`             storage_adapter='chatterbot.storage.SQLStorageAdapter',`
		`             database_uri='sqlite:///campus.sqlite3',`
		`             logic_adapters=[{'import_path': 'chatterbot.logic.BestMatch',`
		`                              'default_response':`
		`                                  'Sorry, I do not know that.',`
		`                              'maximum_similarity_threshold': 0.90}],`
		`             read_only=True)`

3	Train it with question / answer pairs	`trainer = ListTrainer(bot)`
		`trainer.train(['Hi', 'Hello! How can I help?',`
		`              'What are the mess timings?',`
		`                  'Breakfast 7:30-9:00, lunch 12:30-2:00.',`
		`              'Who is the warden?', 'The warden office is in Block A.'])`

4	Ask a question (a close match is enough)	`print(bot.get_response('Hi'))`
		`reply = bot.get_response('what are the mess timing')`
		`print(reply.text)`

5	An unknown question gets the default answer	`print(bot.get_response('tell me a joke'))`

T.No	Task Desc	Program code

6	Chat loop (type bye to stop)	`while True:`
		`  user = input('You: ')`
		`  if user.lower() in ('bye', 'quit'):`
		`  break`
		`  print('Bot:', bot.get_response(user))`

7	Clear the stored knowledge	`bot.storage.drop()`

Note. ChatterBot does not generate sentences: it returns the stored answer whose question is closest to yours. read_only=True stops it from learning from every conversation.

Module 2: Python Libraries (9 hours)

2.1 pandas Series

Syllabus: Module 2: Pandas - Series, Handling Missing Values, Built-in Functions. 14 rows, in running order. Install: pip install pandas numpy.

T.No	Task Desc	Program code
1	Load packages	`import pandas as pd` `import numpy as np`
2	Create a Series with an index	`marks = pd.Series([82, 91, 67, 74], index=['Asha', 'Ravi', 'Meena', 'John'], name='marks')` `print(marks)`

3	From a dictionary, a scalar, an array	`s_dict = pd.Series({'a': 1, 'b': 2, 'c': 3})`
		`zeros = pd.Series(0, index=range(3))`
		`from_np = pd.Series(np.arange(3) * 1.5)`

4	Attributes	`print(marks.index.tolist(), marks.values, marks.dtype, marks.name, marks.shape, marks.size)`
5	Access by label and by position	`print(marks['Ravi'], marks.iloc[0], marks[['Asha', 'John']].tolist(), marks['Ravi':'John'].tolist(), marks.iloc[0:2].tolist())`

6	Vectorised arithmetic aligns on the index	`bonus = pd.Series({'Asha': 5, 'Ravi': 3, 'Zed': 9})`
		`print((marks + bonus).to_dict())`
		`print(marks.add(bonus, fill_value=0).to_dict())`

7	Filter with a Boolean mask	`print(marks[marks > 70].index.tolist(), marks[(marks > 70) & (marks < 90)].tolist())`
8	Statistics	`print(marks.sum(), marks.mean(), marks.median(), round(marks.std(), 2), marks.max(), marks.idxmax(), marks.idxmin(), marks.quantile(0.5))`
9	Sort and rank	`print(marks.sort_values(ascending=False).index.tolist())` `print(marks.sort_index().index.tolist(), marks.rank(ascending=False).tolist())`
10	apply and map	`grade = marks.apply(lambda m: 'A' if m >= 80 else 'B')` `print(grade.to_dict(), marks.map(lambda m: m - 50).tolist())`
11	Missing values	`s = pd.Series([1.0, np.nan, 3.0, np.nan])` `print(s.isna().sum(), s.fillna(s.mean()).tolist(), s.dropna().tolist(), s.ffill().tolist())`
12	Cumulative and rolling calculations	`s2 = pd.Series([1, 2, 3, 4, 5])` `print(s2.cumsum().tolist(), s2.diff().tolist(), s2.pct_change().round(2).tolist(), s2.rolling(2).mean().tolist())`
13	Counting and membership	`colors = pd.Series(['r', 'g', 'r', 'b', 'r'])` `print(colors.value_counts().to_dict(), colors.unique().tolist(), colors.nunique(), colors.isin(['r', 'b']).sum(), marks.between(70, 85).sum())`

14	describe() and conversion to a DataFrame	`print(marks.describe().round(2).to_dict())`
		`frame = marks.to_frame()`
		`print(frame.shape)`

2.2 DataFrame: built-in functions, data operations, filtering, extraction

Syllabus: Module 2: Built-in Functions, Data Operations, Filtering Data in DataFrame, Data Extraction. 19 rows, in running order. Install: pip install pandas numpy.

T.No	Task Desc	Program code
1	Load packages	`import pandas as pd` `import numpy as np`

2	Create the DataFrame	`emp = pd.DataFrame({`
		`'name': ['Asha', 'Ravi', 'Meena', 'John', 'Sara', 'Kabir', 'Dev', 'Zoya'],`
		`'dept': ['IT', 'HR', 'IT', 'Sales', 'HR', 'IT', 'Sales', 'IT'],`
		`'age': [28, 35, 41, 30, 26, 38, 45, 33],`
		`'salary': [52000, 61000, 75000, 48000, 43000, 68000, 72000, 59000],`
		`'city': ['Pune', 'Delhi', 'Pune', 'Mumbai', 'Delhi', 'Pune', 'Mumbai', 'Delhi']})`

3	Inspect: shape, types, summary	`print(emp.shape, emp.columns.tolist())`
		`emp.info()`
		`print(emp.describe().round(1))`

4	Extract columns and rows: [], loc, iloc, at	`print(emp['name'].tolist()[:2])`
		`print(emp.loc[2, 'name'], emp.iloc[2, 0], emp.at[2, 'salary'])`
		`print(emp.loc[1:3, ['name', 'age']])`
		`print(emp.iloc[:3, [0, 3]])`

5	Filter with several conditions	`it_high = emp[(emp['dept'] == 'IT') & (emp['salary'] > 55000)]` `print(it_high['name'].tolist())`
6	isin, between, negation	`print(emp[emp['city'].isin(['Pune', 'Goa'])].shape[0], emp[emp['age'].between(30, 40)]['name'].tolist(), emp[~emp['dept'].isin(['IT'])].shape[0])`

T.No	Task Desc	Program code

7	query() with a variable	`print(emp.query("dept == 'HR' and age > 30")['name'].tolist())`
		`min_salary = 60000`
		`print(emp.query('salary > @min_salary').shape[0])`

8	Top / bottom rows and multi-column sort	`print(emp.nlargest(3, 'salary')['name'].tolist(), emp.nsmallest(2, 'age')['name'].tolist())` `print(emp.sort_values(['dept', 'salary'], ascending=[True, False])['name'].tolist())`

9	Add columns: arithmetic, np.where, map, assign	`emp['bonus'] = emp['salary'] * 0.1`
		`emp['band'] = np.where(emp['salary'] >= 60000, 'high', 'normal')`
		`emp['initial'] = emp['name'].map(lambda n: n[0])`
		`emp = emp.assign(annual=lambda d: d['salary'] * 12)`
		`print(emp[['name', 'bonus', 'band', 'initial']].head(3))`

10	Row-wise apply	`emp['label'] = emp.apply(lambda row: f"{row['name']} ({row['dept']})", axis=1)` `print(emp['label'].iloc[0])`
11	Drop and rename	`slim = emp.drop(columns=['bonus', 'label', 'initial', 'annual']).rename(columns={'salary': 'pay'})` `print(slim.columns.tolist())`
12	groupby with named aggregations	`summary = emp.groupby('dept').agg(people=('name', 'count'), avg_salary=('salary', 'mean'), oldest=('age', 'max'))` `print(summary)`

13	groupby transform and filter	`emp['dept_avg'] = emp.groupby('dept')['salary'].transform('mean')`
		`emp['above_avg'] = emp['salary'] > emp['dept_avg']`
		`big = emp.groupby('dept').filter(lambda g: len(g) >= 3)`
		`print(emp['above_avg'].sum(), big['dept'].unique().tolist())`

14	Counts, unique values, duplicates	`print(emp['city'].value_counts(normalize=True).round(2).to_dict(), emp['city'].nunique(), emp.duplicated().sum())`

15	Rank and quantile bins	`emp['rank'] = emp['salary'].rank(ascending=False).astype(int)`
		`emp['tier'] = pd.qcut(emp['salary'], 3, labels=['low', 'mid', 'top'])`
		`print(emp[['name', 'rank', 'tier']].sort_values('rank').head(3))`

16	Use a column as the index	`by_name = emp.set_index('name')`
		`print(by_name.loc['Ravi', 'salary'])`
		`back = by_name.reset_index()`

17	Replace values, category type	`emp['dept'] = emp['dept'].replace({'IT': 'Tech'}).astype('category')` `print(emp['dept'].cat.categories.tolist())`
18	Iterate over rows	`for row in emp.head(2).itertuples():` `  print(row.name, row.salary)`
19	Sampling and quick tests	`print(emp.sample(3, random_state=1).shape, emp.tail(1)['name'].tolist())` `print((emp['salary'] > 40000).all(), (emp['age'] > 44).any(), (emp['age'] > 30).sum())`

Note. Use loc for labels and iloc for positions. Combine conditions with & | ~ and put each condition in brackets.

2.3 Working with text data in pandas

Syllabus: Module 2: Working with Text Data. 11 rows, in running order. Install: pip install pandas.

T.No	Task Desc	Program code
1	Load packages and a text column	`import pandas as pd` `names = pd.Series([' Asha Rao ', 'ravi KUMAR', 'Meena  Iyer', None, 'John Smith'])`

2	Case, whitespace, length	`clean = names.str.strip().str.title()`
		`print(clean.tolist())`
		`print(names.str.len().tolist(), names.str.lower().iloc[1], names.str.upper().iloc[0])`

3	Search: contains, startswith, endswith	`print(clean.str.contains('Rao').tolist())` `print(clean.str.startswith('J', na=False).tolist(), clean.str.endswith('r').fillna(False).tolist())`
4	Replace (with a regular expression)	`print(names.str.replace(r'\s+', ' ', regex=True).str.strip().tolist())`

5	Split into columns	`parts = clean.str.split(expand=True)`
		`parts.columns = ['first', 'last']`
		`print(parts)`

6	Extract with capture groups	`emails = pd.Series(['asha@vit.ac.in', 'ravi@gmail.com', 'bad.address'])` `print(emails.str.extract(r'(\w+)@([\w.]+)'))`
7	Count, find, slice, get	`print(emails.str.count('a').tolist(), emails.str.find('@').tolist(), emails.str[:4].tolist(), emails.str.split('@').str[0].tolist())`

8	Join Series, pad numbers	`first = pd.Series(['A', 'B'])`
		`last = pd.Series(['X', 'Y'])`
		`print(first.str.cat(last, sep=' ').tolist())`
		`ids = pd.Series(['7', '42', '105'])`
		`print(ids.str.zfill(4).tolist(), ids.str.pad(5, side='left', fillchar='*').tolist())`

T.No	Task Desc	Program code

9	Text columns inside a DataFrame	`reviews = pd.DataFrame({'review': ['Great product!', 'Bad quality', 'GREAT value', 'ok']})`
		`reviews['positive'] = reviews['review'].str.lower().str.contains('great')`
		`reviews['n_words'] = reviews['review'].str.split().str.len()`
		`reviews['clean'] = reviews['review'].str.lower().str.replace(r'[^a-z ]', '', regex=True)`
		`print(reviews)`

10	Dummy variables from delimited text	`tags = pd.Series(['a|b', 'b|c', 'a'])` `print(tags.str.get_dummies())`
11	Missing text: na handling	`print(names.isna().sum(), names.fillna('unknown').str.title().tolist())`

2.4 Merging DataFrames: merge, join, concat, pivot

Syllabus: Module 2: Merging DataFrames. 12 rows, in running order. Install: pip install pandas.

T.No	Task Desc	Program code

1	Two related tables	`import pandas as pd`
		`students = pd.DataFrame({'id': [1, 2, 3, 4], 'name': ['Asha', 'Ravi', 'Meena', 'John']})`
		`marks = pd.DataFrame({'id': [2, 3, 5], 'score': [88, 92, 70]})`

2	Inner join (matching keys only)	`inner = pd.merge(students, marks, on='id', how='inner')` `print(inner)`

3	Left, right and outer joins	`left = students.merge(marks, on='id', how='left')`
		`right = students.merge(marks, on='id', how='right')`
		`outer = students.merge(marks, on='id', how='outer', indicator=True)`
		`print(len(inner), len(left), len(right), len(outer))`
		`print(outer['_merge'].value_counts().to_dict())`

4	Key columns with different names	`marks2 = marks.rename(columns={'id': 'student_id'})`
		`m2 = pd.merge(students, marks2, left_on='id', right_on='student_id')`
		`print(m2.columns.tolist())`

5	Overlapping column names: suffixes	`a = pd.DataFrame({'id': [1, 2], 'v': [10, 20]})`
		`b = pd.DataFrame({'id': [1, 2], 'v': [1, 2]})`
		`print(a.merge(b, on='id', suffixes=('_old', '_new')).columns.tolist())`

6	Merge on several keys	`x = pd.DataFrame({'dept': ['IT', 'IT', 'HR'], 'year': [2025, 2026, 2026], 'budget': [5, 6, 3]})`
		`y = pd.DataFrame({'dept': ['IT', 'HR'], 'year': [2026, 2026], 'spend': [4, 2]})`
		`print(x.merge(y, on=['dept', 'year'], how='left'))`

7	join() works on the index	`s_idx = students.set_index('id')`
		`m_idx = marks.set_index('id')`
		`print(s_idx.join(m_idx, how='left'))`

8	concat: stack rows or put columns side by side	`more = pd.DataFrame({'id': [9], 'name': ['Zed']})`
		`all_students = pd.concat([students, more], ignore_index=True)`
		`side = pd.concat([students, marks], axis=1)`
		`print(all_students.shape, side.shape)`

9	concat with keys (hierarchical index)	`stacked = pd.concat([students, more], keys=['2025', '2026'])` `print(stacked.loc['2026'])`
10	Anti-join: rows with no match	`only_students = students.merge(marks, on='id', how='left', indicator=True).query("_merge == 'left_only'")` `print(only_students['name'].tolist())`

11	Check the relationship with validate=	`try:`
		`  pd.merge(students, pd.concat([marks, marks]), on='id', validate='one_to_one')`
		`except pd.errors.MergeError as e:`
		`  print('MergeError:', e)`

12	Reshape: pivot_table and melt	`sales = pd.DataFrame({'region': ['N', 'N', 'S', 'S'], 'q': ['Q1', 'Q2', 'Q1', 'Q2'], 'amt': [10, 20, 30, 40]})`
		`wide = sales.pivot_table(index='region', columns='q', values='amt', aggfunc='sum')`
		`long = wide.reset_index().melt(id_vars='region', var_name='q', value_name='amt')`
		`print(wide)`
		`print(long.shape)`

Note. merge = SQL join on columns, join = merge on the index, concat = stack along an axis. how='inner' keeps matches only, 'left' keeps every left row (NaN where no match).

2.5 SciPy: statistics, optimisation, integration, interpolation, linear algebra

Syllabus: Module 2: SciPy. 14 rows, in running order. Install: pip install scipy numpy.

T.No	Task Desc	Program code
1	Load packages	`import numpy as np` `from scipy import stats, optimize, integrate, interpolate, linalg`

2	Descriptive statistics	`data = np.array([12, 15, 14, 10, 18, 20, 11, 16])`
		`print(stats.describe(data))`
		`print(stats.zscore(data).round(2), round(stats.skew(data), 3))`

3	Normal distribution: cdf, ppf, pdf	`print(round(stats.norm.cdf(1.96), 4), round(stats.norm.ppf(0.975), 4),` `  round(stats.norm.pdf(0), 4))`

4	Two-sample t-test	`a = np.array([5.1, 4.9, 5.6, 5.8, 6.0, 5.5])`
		`b = np.array([6.2, 6.4, 5.9, 6.8, 7.0, 6.5])`
		`t, p = stats.ttest_ind(a, b)`
		`print(round(t, 3), round(p, 4), 'significant' if p < 0.05 else`
		`  'not significant')`

5	One-sample and paired t-tests	`print(round(stats.ttest_1samp(a, 5.0).pvalue, 4))` `print(round(stats.ttest_rel(a, b).pvalue, 5))`

6	Correlation: Pearson and Spearman	`x = np.arange(1, 9)`
		`y = np.array([2.1, 3.9, 6.2, 7.8, 10.1, 12.2, 13.8, 16.1])`
		`r, p = stats.pearsonr(x, y)`
		`rho, _ = stats.spearmanr(x, y)`
		`print(round(r, 4), round(rho, 4))`

7	Simple linear regression	`res = stats.linregress(x, y)` `print(round(res.slope, 3), round(res.intercept, 3), round(res.rvalue ** 2, 4))`

8	Chi-square test of independence	`table = np.array([[30, 10], [20, 40]])`
		`chi2, p, dof, expected = stats.chi2_contingency(table)`
		`print(round(chi2, 2), round(p, 5), dof)`

9	Minimise a function	`res = optimize.minimize(lambda v: (v[0] - 3) ** 2 + (v[1] + 1) ** 2, x0=[0, 0])` `print(res.x.round(3), res.success)`
10	Find a root	`root = optimize.root_scalar(lambda x: x ** 3 - 2 * x - 5, bracket=[2, 3])` `print(round(root.root, 4))`

11	Fit a curve	`def model(x, a, b):`
		`  return a * np.exp(b * x)`
		
		`xs = np.linspace(0, 2, 20)`
		`ys = 2.0 * np.exp(0.8 * xs)`
		`popt, _ = optimize.curve_fit(model, xs, ys)`
		`print(popt.round(3))`

12	Numerical integration	`area, err = integrate.quad(lambda x: x ** 2, 0, 3)` `print(round(area, 4))`

13	Interpolation	`f_lin = interpolate.interp1d([0, 1, 2], [0, 10, 40])`
		`f_quad = interpolate.interp1d([0, 1, 2], [0, 10, 40], kind='quadratic')`
		`print(f_lin(1.5), f_quad(1.5))`

14	Linear algebra: solve, det, inverse, eigenvalues	`A = np.array([[3, 1], [1, 2]])`
		`rhs = np.array([9, 8])`
		`print(linalg.solve(A, rhs), round(linalg.det(A), 2),`
		`  linalg.inv(A).round(2).tolist())`
		`vals, vecs = linalg.eig(A)`
		`print(np.sort(vals.real).round(3))`

2.6 Scrapy: a spider with items, a pipeline and pagination

Syllabus: Module 2: Data Mining - Scrapy. 8 rows, in running order. Needs Scrapy. Install: pip install scrapy.

T.No	Task Desc	Program code

1	Load packages	`import json`
		`import scrapy`
		`from scrapy.crawler import CrawlerProcess`

2	Item: the fields to collect	`class Quote(scrapy.Item):`
		`  text = scrapy.Field()`
		`  author = scrapy.Field()`
		`  tags = scrapy.Field()`

3	Item pipeline: clean every item	`class CleanPipeline:`
		`  def process_item(self, item, spider):`
		`  item['text'] = item['text'].strip('\u201c\u201d')`
		`  return item`

T.No	Task Desc	Program code

4	Spider: start URL, CSS selectors, next page, settings	`class QuotesSpider(scrapy.Spider):`
		`  name = 'quotes'`
		`  start_urls = ['https://quotes.toscrape.com/']`
		`  custom_settings = {`
		`  'LOG_LEVEL': 'ERROR',`
		`  'ROBOTSTXT_OBEY': True,`
		`  'DOWNLOAD_DELAY': 1,`
		`  'FEEDS': {'quotes.json': {'format': 'json', 'overwrite': True,`
		`  'encoding': 'utf8'}},`
		`  'ITEM_PIPELINES': {'__main__.CleanPipeline': 300},`
		`  }`
		
		`  def parse(self, response):`
		`  for q in response.css('div.quote'):`
		`  item = Quote()`
		`  item['text'] = q.css('span.text::text').get()`
		`  item['author'] = q.css('small.author::text').get()`
		`  item['tags'] = q.css('a.tag::text').getall()`
		`  yield item`
		`  next_page = response.css('li.next a::attr(href)').get()`
		`  if next_page:`
		`  yield response.follow(next_page, callback=self.parse)`

5	Selectors on their own: CSS and XPath	`from scrapy import Selector`
		`sel = Selector(text='<div class="quote"><span class="text">Hi</span><a href="/x`
		`  ">link</a></div>')`
		`print(sel.css('span.text::text').get(),`
		`  sel.xpath('//span[@class="text"]/text()').get(),`
		`  sel.css('a::attr(href)').get())`

6	Run the crawl	`process = CrawlerProcess()`
		`process.crawl(QuotesSpider)`
		`process.start()`

7	Read the scraped file	`with open('quotes.json', encoding='utf-8') as f:`
		`  items = json.load(f)`
		`print(len(items), [i['author'] for i in items])`

8	Terminal commands for a full project
	(not run: shell commands)
		`# scrapy startproject quotes_project`
		`# scrapy genspider quotes quotes.toscrape.com`
		`# scrapy crawl quotes -o quotes.csv`
		`# scrapy shell 'https://quotes.toscrape.com'`

Note. Scrapy is a framework (scheduler, downloader, pipelines) for crawling many pages, Beautiful Soup only parses one HTML string. The page is fetched with response.follow, so relative links work. Verified on two local pages that have the same HTML structure as quotes.toscrape.com (the live site was not contacted).

2.7 scikit-learn: the supervised learning workflow

Syllabus: Module 2: Scikit-Learn. 14 rows, in running order. Install: pip install scikit-learn joblib.

T.No	Task Desc	Program code

1	Load packages	`import numpy as np`
		`from sklearn.datasets import load_iris, load_diabetes`
		`from sklearn.model_selection import train_test_split, cross_val_score,`
		`  GridSearchCV`
		`from sklearn.preprocessing import StandardScaler`
		`from sklearn.pipeline import Pipeline`
		`from sklearn.linear_model import LinearRegression, LogisticRegression`
		`from sklearn.tree import DecisionTreeClassifier, export_text`
		`from sklearn.ensemble import RandomForestClassifier`
		`from sklearn.neighbors import KNeighborsClassifier`
		`from sklearn.metrics import accuracy_score, confusion_matrix,`
		`  classification_report, r2_score, mean_squared_error`

2	Features, labels and a train / test split	`X, y = load_iris(return_X_y=True)`
		`X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,`
		`  random_state=42, stratify=y)`
		`print(X_train.shape, X_test.shape)`

3	Scale: learn the mean and std on the training set only	`scaler = StandardScaler().fit(X_train)` `Xs_train, Xs_test = scaler.transform(X_train), scaler.transform(X_test)`

4	Logistic regression: fit, predict, accuracy	`clf = LogisticRegression(max_iter=200).fit(Xs_train, y_train)`
		`pred = clf.predict(Xs_test)`
		`print(round(accuracy_score(y_test, pred), 3))`

5	Confusion matrix and classification report	`print(confusion_matrix(y_test, pred))`
		`print(classification_report(y_test, pred,`
		`  target_names=load_iris().target_names))`

6	Decision tree and its rules	`tree = DecisionTreeClassifier(max_depth=2, random_state=0).fit(X_train,`
		`  y_train)`
		`print(export_text(tree, feature_names=load_iris().feature_names))`
		`print(round(tree.score(X_test, y_test), 3))`

T.No	Task Desc	Program code
7	Random forest and feature importance	`rf = RandomForestClassifier(n_estimators=100, random_state=0).fit(X_train, y_train)` `print(round(rf.score(X_test, y_test), 3), rf.feature_importances_.round(2))`

8	k-nearest neighbours: effect of k	`for k in (1, 5, 15):`
		`  knn = KNeighborsClassifier(n_neighbors=k).fit(Xs_train, y_train)`
		`  print(k, round(knn.score(Xs_test, y_test), 3))`

9	Linear regression: R-squared and RMSE	`Xd, yd = load_diabetes(return_X_y=True)`
		`Xd_tr, Xd_te, yd_tr, yd_te = train_test_split(Xd, yd, test_size=0.2, random_state=0)`
		`lin = LinearRegression().fit(Xd_tr, yd_tr)`
		`pred_d = lin.predict(Xd_te)`
		`print(round(r2_score(yd_te, pred_d), 3), round(mean_squared_error(yd_te, pred_d) ** 0.5, 1))`
		`print(lin.coef_.shape, round(lin.intercept_, 1))`

10	Cross-validation	`scores = cross_val_score(LogisticRegression(max_iter=200), Xs_train, y_train, cv=5)` `print(scores.round(3), round(scores.mean(), 3))`

11	Pipeline: scaler and model in one object	`pipe = Pipeline([('scale', StandardScaler()), ('model', LogisticRegression(max_iter=200))])`
		`pipe.fit(X_train, y_train)`
		`print(round(pipe.score(X_test, y_test), 3))`

12	Grid search over a hyper-parameter	`knn_pipe = Pipeline([('scale', StandardScaler()), ('model', KNeighborsClassifier())])`
		`grid = GridSearchCV(knn_pipe, {'model__n_neighbors': [1, 3, 5, 9]}, cv=5)`
		`grid.fit(X_train, y_train)`
		`print(grid.best_params_, round(grid.best_score_, 3))`

13	Class probabilities	`print(pipe.predict_proba([[6.0, 3.0, 4.8, 1.8]]).round(3))`

14	Save and load the trained model	`import joblib`
		`joblib.dump(pipe, 'iris_model.joblib')`
		`model = joblib.load('iris_model.joblib')`
		`print(model.predict([[5.1, 3.5, 1.4, 0.2]]))`

Note. Fit the scaler on the training data only, otherwise information from the test set leaks into the model. Classification metrics: accuracy, precision, recall, F1. Regression metrics: R-squared, MSE / RMSE.

2.8 XGBoost: gradient-boosted trees

Syllabus: Module 2: XGBoost. 7 rows, in running order. Needs TensorFlow / Keras / PyTorch / XGBoost. Install: pip install xgboost scikit-learn.

T.No	Task Desc	Program code

1	Load packages and split the data	`import numpy as np`
		`import xgboost as xgb`
		`from sklearn.datasets import load_breast_cancer, load_diabetes`
		`from sklearn.model_selection import train_test_split`
		`data = load_breast_cancer()`
		`X_train, X_test, y_train, y_test = train_test_split(data.data, data.target, test_size=0.3, random_state=0, stratify=data.target)`

2	Classifier: fit and score	`clf = xgb.XGBClassifier(n_estimators=100, max_depth=3, learning_rate=0.1, random_state=0)`
		`clf.fit(X_train, y_train)`
		`print(round(clf.score(X_test, y_test), 3))`

3	Probabilities and the most important features	`print(clf.predict_proba(X_test[:2]).round(3))`
		`top = np.argsort(clf.feature_importances_)[::-1][:3]`
		`print([str(data.feature_names[i]) for i in top])`

4	Early stopping on a validation set	`X_tr, X_val, y_tr, y_val = train_test_split(X_train, y_train, test_size=0.25, random_state=0, stratify=y_train)`
		`early = xgb.XGBClassifier(n_estimators=500, learning_rate=0.1, early_stopping_rounds=10, eval_metric='logloss', random_state=0)`
		`early.fit(X_tr, y_tr, eval_set=[(X_val, y_val)], verbose=False)`
		`print(early.best_iteration < 500)`

5	Regressor on a regression data set	`Xd, yd = load_diabetes(return_X_y=True)`
		`Xd_tr, Xd_te, yd_tr, yd_te = train_test_split(Xd, yd, test_size=0.2, random_state=0)`
		`reg = xgb.XGBRegressor(n_estimators=200, max_depth=3, learning_rate=0.05, random_state=0).fit(Xd_tr, yd_tr)`
		`rmse = float(np.sqrt(np.mean((reg.predict(Xd_te) - yd_te) ** 2)))`
		`print(round(rmse, 1))`

6	The native API: DMatrix and train()	`dtrain = xgb.DMatrix(X_train, label=y_train)`
		`params = {'objective': 'binary:logistic', 'max_depth': 3, 'eta': 0.1}`
		`booster = xgb.train(params, dtrain, num_boost_round=50)`
		`print(booster.predict(xgb.DMatrix(X_test))[:3].round(3))`

7	Save and load a model	`clf.save_model('model.json')`
		`clf2 = xgb.XGBClassifier()`
		`clf2.load_model('model.json')`
		`print(clf2.score(X_test, y_test) == clf.score(X_test, y_test))`

Note. Boosting builds trees one after another and each new tree fits the errors of the previous ones. Lower learning_rate needs more trees. Never use the test set for early stopping.

2.9 Keras: a neural network with the Sequential API

Syllabus: Module 2: Keras. 11 rows, in running order. Needs TensorFlow / Keras / PyTorch / XGBoost. Install: pip install keras tensorflow scikit-learn.

T.No	Task Desc	Program code

1	Load packages, fix the random seed	`import os`
		`os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'`
		`import numpy as np`
		`import keras`
		`from keras import layers`
		`from sklearn.datasets import load_iris`
		`from sklearn.model_selection import train_test_split`
		`from sklearn.preprocessing import StandardScaler`
		`keras.utils.set_random_seed(0)`

2	Prepare the data	`X, y = load_iris(return_X_y=True)`
		`X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,`
		`  random_state=0, stratify=y)`
		`scaler = StandardScaler().fit(X_train)`
		`X_train, X_test = scaler.transform(X_train), scaler.transform(X_test)`

3	Define the layers	`model = keras.Sequential([`
		`  layers.Input(shape=(4,)),`
		`  layers.Dense(16, activation='relu'),`
		`  layers.Dense(8, activation='relu'),`
		`  layers.Dense(3, activation='softmax'),`
		`])`

4	Show the architecture and parameter count	`model.summary()` `print(model.count_params())`
5	Compile: optimiser, loss, metric	`model.compile(optimizer=keras.optimizers.Adam(0.01),` `  loss='sparse_categorical_crossentropy', metrics=['accuracy'])`

6	Train	`history = model.fit(X_train, y_train, epochs=40, batch_size=16,`
		`  validation_split=0.2, verbose=0)`
		`print(round(history.history['loss'][0], 3), '->',`
		`  round(history.history['loss'][-1], 3))`

7	Evaluate on the test set	`loss, acc = model.evaluate(X_test, y_test, verbose=0)` `print(round(acc, 3))`
8	Predict	`probs = model.predict(X_test[:3], verbose=0)` `print(probs.shape, probs.argmax(axis=1), y_test[:3])`

9	Stop early when validation loss stops improving	`stop = keras.callbacks.EarlyStopping(monitor='val_loss', patience=5,`
		`  restore_best_weights=True)`
		`h2 = model.fit(X_train, y_train, epochs=100, validation_split=0.2,`
		`  callbacks=[stop], verbose=0)`
		`print(len(h2.history['loss']) <= 100)`

10	The functional API	`inp = layers.Input(shape=(4,))`
		`hidden = layers.Dense(8, activation='relu')(inp)`
		`out = layers.Dense(3, activation='softmax')(hidden)`
		`func = keras.Model(inp, out)`
		`print(func.count_params())`

11	Save and reload the model	`model.save('iris.keras')`
		`reloaded = keras.models.load_model('iris.keras')`
		`print(np.allclose(model.predict(X_test[:2], verbose=0),`
		`  reloaded.predict(X_test[:2], verbose=0)))`

Note. Parameters of a Dense layer = inputs x units + units (4 x 16 + 16 = 80). softmax for several classes, sigmoid for two, linear output for regression. sparse_categorical_crossentropy takes integer labels, categorical_crossentropy takes one-hot vectors.

2.10 TensorFlow: tensors, gradients, tf.function, tf.data

Syllabus: Module 2: TensorFlow. 8 rows, in running order. Needs TensorFlow / Keras / PyTorch / XGBoost. Install: pip install tensorflow.

T.No	Task Desc	Program code

1	Load packages	`import os`
		`os.environ['TF_CPP_MIN_LOG_LEVEL'] = '3'`
		`import numpy as np`
		`import tensorflow as tf`

2	Tensors and operations	`a = tf.constant([[1.0, 2.0], [3.0, 4.0]])`
		`b = tf.ones((2, 2))`
		`print(a.shape, a.dtype.name, (a + b).numpy().tolist(), tf.matmul(a,`
		`  a).numpy().tolist())`

3	Variables and reductions	`w = tf.Variable([1.0, 2.0])`
		`w.assign_add([0.5, 0.5])`
		`print(w.numpy(), tf.reduce_sum(w).numpy(), tf.reduce_mean(a, axis=0).numpy())`

T.No	Task Desc	Program code

4	Automatic differentiation with GradientTape	`x = tf.constant(3.0)`
		`with tf.GradientTape() as tape:`
		`  tape.watch(x)`
		`  y = x ** 2`
		`print(tape.gradient(y, x).numpy())`

5	Gradient descent by hand: fit y = 2x + 1	`w, b = tf.Variable(0.0), tf.Variable(0.0)`
		`xs = tf.constant([1.0, 2.0, 3.0, 4.0])`
		`ys = 2.0 * xs + 1.0`
		`opt = tf.keras.optimizers.SGD(learning_rate=0.05)`
		`for _ in range(300):`
		`  with tf.GradientTape() as tape:`
		`  loss = tf.reduce_mean((w * xs + b - ys) ** 2)`
		`  grads = tape.gradient(loss, [w, b])`
		`  opt.apply_gradients(zip(grads, [w, b]))`
		`print(round(float(w.numpy()), 1), round(float(b.numpy()), 1))`

6	tf.function: compile a Python function into a graph	`@tf.function`
		`def square_sum(v):`
		`  return tf.reduce_sum(v * v)`
		
		`print(square_sum(tf.constant([1.0, 2.0, 3.0])).numpy())`

7	tf.data: build an input pipeline	`ds = tf.data.Dataset.from_tensor_slices((np.arange(6, dtype='float32'),`
		`  np.arange(6) % 2)).shuffle(6, seed=0).batch(3)`
		`for xb, yb in ds:`
		`  print(xb.shape, yb.shape)`

8	A tf.keras model trained from the pipeline	`xs_np = np.linspace(-1, 1, 64).astype('float32')`
		`ds = tf.data.Dataset.from_tensor_slices((xs_np, 3 * xs_np + 2)).batch(16)`
		`net = tf.keras.Sequential([tf.keras.layers.Input(shape=(1,)),`
		`  tf.keras.layers.Dense(1)])`
		`net.compile(optimizer=tf.keras.optimizers.Adam(0.1), loss='mse')`
		`hist = net.fit(ds.map(lambda a, b: (tf.expand_dims(a, -1), b)), epochs=30,`
		`  verbose=0)`
		`print(hist.history['loss'][-1] < hist.history['loss'][0])`

2.11 PyTorch: tensors, autograd, nn.Module, training loop

Syllabus: Module 2: PyTorch. 11 rows, in running order. Needs TensorFlow / Keras / PyTorch / XGBoost. Install: pip install torch scikit-learn.

T.No	Task Desc	Program code

1	Load packages	`import torch`
		`from torch import nn`
		`from torch.utils.data import TensorDataset, DataLoader`
		`from sklearn.datasets import load_iris`
		`from sklearn.preprocessing import StandardScaler`

2	Tensors	`t = torch.tensor([[1.0, 2.0], [3.0, 4.0]])`
		`print(t.shape, t.dtype, (t @ t).tolist(), t.mean().item(), t.view(4).tolist(),`
		`  torch.zeros(2, 3).shape)`

3	NumPy interoperability	`arr = t.numpy()`
		`back = torch.from_numpy(arr)`
		`print(type(arr).__name__, back.dtype)`

4	Autograd: gradient of y = x^2 + 2x at x = 3	`x = torch.tensor(3.0, requires_grad=True)`
		`y = x ** 2 + 2 * x`
		`y.backward()`
		`print(x.grad.item())`

5	Data as tensors and a DataLoader	`Xn, yn = load_iris(return_X_y=True)`
		`X = torch.tensor(StandardScaler().fit_transform(Xn), dtype=torch.float32)`
		`y = torch.tensor(yn, dtype=torch.long)`
		`loader = DataLoader(TensorDataset(X, y), batch_size=16, shuffle=True)`
		`print(len(loader))`

6	The model as an nn.Module	`class Net(nn.Module):`
		`  def __init__(self):`
		`  super().__init__()`
		`  self.layers = nn.Sequential(nn.Linear(4, 16), nn.ReLU(), nn.Linear(16,`
		`  3))`
		
		`  def forward(self, x):`
		`  return self.layers(x)`
		
		`torch.manual_seed(0)`
		`net = Net()`

7	Loss function and optimiser	`loss_fn = nn.CrossEntropyLoss()` `optimizer = torch.optim.Adam(net.parameters(), lr=0.01)`

8	Training loop: zero_grad, forward, loss, backward, step	`for epoch in range(30):`
		`  for xb, yb in loader:`
		`  optimizer.zero_grad()`
		`  loss = loss_fn(net(xb), yb)`
		`  loss.backward()`
		`  optimizer.step()`
		`print(round(loss.item(), 3))`

T.No	Task Desc	Program code

9	Evaluate without tracking gradients	`net.eval()`
		`with torch.no_grad():`
		`  acc = (net(X).argmax(dim=1) == y).float().mean().item()`
		`print(acc > 0.9)`

10	Save and load the weights	`torch.save(net.state_dict(), 'net.pt')`
		`net2 = Net()`
		`net2.load_state_dict(torch.load('net.pt'))`
		`print(torch.equal(net(X), net2(X)))`

11	Count the parameters	`print(sum(p.numel() for p in net.parameters()))`

Note. PyTorch adds gradients up: call optimizer.zero_grad() every step. CrossEntropyLoss expects raw scores (logits) and int64 class labels.

2.12 Folium: interactive maps

Syllabus: Module 2: Data Visualization - Folium. 12 rows, in running order. Install: pip install folium.

T.No	Task Desc	Program code

1	Load packages	`import folium`
		`from folium.plugins import MarkerCluster, HeatMap`
		`from pathlib import Path`

2	Base map: centre, zoom, tiles	`m = folium.Map(location=[12.9692, 79.1559], zoom_start=7,` `  tiles='OpenStreetMap')`
3	Marker with popup, tooltip, icon	`folium.Marker([12.9692, 79.1559], popup='VIT Vellore', tooltip='Click me',` `  icon=folium.Icon(color='blue', icon='info-sign')).add_to(m)`

4	Markers from a dictionary of cities	`cities = {'Vellore': (12.9165, 79.1325), 'Chennai': (13.0827, 80.2707),`
		`  'Bengaluru': (12.9716, 77.5946)}`
		`for name, (lat, lon) in cities.items():`
		`  folium.Marker([lat, lon], popup=name).add_to(m)`

5	CircleMarker (pixels) and Circle (metres)	`folium.CircleMarker([13.0827, 80.2707], radius=12, color='red', fill=True,`
		`  fill_opacity=0.6).add_to(m)`
		`folium.Circle([12.9165, 79.1325], radius=20000, color='green').add_to(m)`

6	A route with PolyLine	`folium.PolyLine(list(cities.values()), color='purple', weight=3,` `  tooltip='route').add_to(m)`

7	Layers with FeatureGroup and LayerControl	`fg = folium.FeatureGroup(name='Landmarks')`
		`folium.Marker([12.9716, 77.5946], tooltip='Cubbon Park').add_to(fg)`
		`fg.add_to(m)`
		`folium.LayerControl().add_to(m)`

8	Cluster many markers	`cluster = MarkerCluster().add_to(m)`
		`for i in range(10):`
		`  folium.Marker([12.9 + i * 0.01, 79.1 + i * 0.01]).add_to(cluster)`

9	Heat map from [lat, lon, weight]	`HeatMap([[12.97, 79.15, 0.9], [13.08, 80.27, 0.6], [12.97, 77.59,` `  0.8]]).add_to(m)`

10	Draw a region with GeoJson	`zone = {'type': 'FeatureCollection', 'features': [`
		`  {'type': 'Feature', 'properties': {'name': 'Zone A'},`
		`  'geometry': {'type': 'Polygon', 'coordinates': [[[79.0, 12.8], [79.3,`
		`  12.8], [79.3, 13.1], [79.0, 13.1], [79.0, 12.8]]]}}]}`
		`folium.GeoJson(zone, name='zone',`
		`  tooltip=folium.GeoJsonTooltip(fields=['name'])).add_to(m)`

11	Choropleth: colour regions by a value	`import pandas as pd`
		`geo = {'type': 'FeatureCollection', 'features': [`
		`  {'type': 'Feature', 'id': 'A', 'properties': {}, 'geometry': {'type':`
		`  'Polygon', 'coordinates': [[[78, 12], [79, 12], [79, 13], [78, 13],`
		`  [78, 12]]]}},`
		`  {'type': 'Feature', 'id': 'B', 'properties': {}, 'geometry': {'type':`
		`  'Polygon', 'coordinates': [[[79, 12], [80, 12], [80, 13], [79, 13],`
		`  [79, 12]]]}}]}`
		`values = pd.DataFrame({'zone': ['A', 'B'], 'population': [120, 340]})`
		`folium.Choropleth(geo_data=geo, data=values, columns=['zone', 'population'],`
		`  key_on='feature.id', fill_color='YlGn', legend_name='Population').add_to(m)`

12	Save as an HTML file	`m.save('map.html')`
		`html = Path('map.html').read_text(encoding='utf-8')`
		`print(len(html) > 1000, 'leaflet' in html.lower())`

Note. Folium builds a Leaflet web page: the map tiles are fetched by the browser, so creating and saving the map needs no internet. Coordinates are [latitude, longitude].

2.13 More Matplotlib and seaborn plots

Syllabus: Module 2: Data Visualization - Matplotlib, Seaborn. 11 rows, in running order. Install: pip install matplotlib seaborn pandas.

T.No	Task Desc	Program code

1	Load packages and data	`import numpy as np`
		`import pandas as pd`
		`import matplotlib.pyplot as plt`
		`import seaborn as sns`
		`rng = np.random.default_rng(0)`
		`days = np.arange(1, 8)`
		`temp = np.array([29, 31, 30, 33, 32, 28, 27])`
		`df = pd.DataFrame({'group': rng.choice(['A', 'B', 'C'], 90), 'x':`
		`  rng.normal(50, 10, 90)})`
		`df['y'] = df['x'] * 0.8 + rng.normal(0, 5, 90)`

2	Line plot with markers, labels, grid	`plt.figure(figsize=(6, 3.5))`
		`plt.plot(days, temp, marker='o', color='tab:red', label='temperature')`
		`plt.xlabel('Day')`
		`plt.ylabel('Temperature (C)')`
		`plt.title('Daily temperature')`
		`plt.grid(True, alpha=0.3)`
		`plt.legend()`
		`plt.savefig('line.png', dpi=100)`

3	Several lines and an annotation	`plt.figure()`
		`plt.plot(days, temp, label='city A')`
		`plt.plot(days, temp - 3, '--', label='city B')`
		`plt.annotate('peak', xy=(4, 33), xytext=(5, 33.5),`
		`  arrowprops=dict(arrowstyle='->'))`
		`plt.legend()`
		`plt.savefig('two_lines.png')`

4	Bar and horizontal bar with value labels	`fig, ax = plt.subplots()`
		`bars = ax.bar(['Q1', 'Q2', 'Q3'], [120, 150, 90], color=['#4c72b0', '#55a868',`
		`  '#c44e52'])`
		`ax.bar_label(bars)`
		`fig.savefig('bar.png')`
		`plt.figure()`
		`plt.barh(['x', 'y'], [3, 5])`
		`plt.savefig('barh.png')`

5	Pie chart	`plt.figure()`
		`plt.pie([45, 30, 25], labels=['IT', 'HR', 'Sales'], autopct='%1.1f%%',`
		`  explode=(0.05, 0, 0), startangle=90)`
		`plt.axis('equal')`
		`plt.savefig('pie.png')`

6	Histogram with a density curve	`plt.figure()`
		`sns.histplot(df['x'], bins=12, kde=True)`
		`plt.savefig('hist.png')`

7	A grid of subplots	`fig, axes = plt.subplots(2, 2, figsize=(8, 6), sharex=False)`
		`axes[0, 0].hist(df['x'])`
		`axes[0, 1].scatter(df['x'], df['y'], s=10)`
		`axes[1, 0].boxplot([df[df['group'] == g]['y'] for g in 'ABC'])`
		`axes[1, 1].plot(days, temp)`
		`fig.suptitle('Dashboard')`
		`fig.tight_layout()`
		`fig.savefig('grid.png')`

8	Box plot and violin plot by group	`plt.figure()`
		`sns.boxplot(data=df, x='group', y='y')`
		`plt.figure()`
		`sns.violinplot(data=df, x='group', y='y')`
		`plt.savefig('violin.png')`

9	Counts, regression line, pair plot	`plt.figure()`
		`sns.countplot(data=df, x='group')`
		`plt.figure()`
		`sns.regplot(data=df, x='x', y='y')`
		`g = sns.pairplot(df, hue='group')`
		`g.savefig('pair.png')`

10	Two y axes	`fig, ax1 = plt.subplots()`
		`ax1.plot(days, temp, color='tab:red')`
		`ax2 = ax1.twinx()`
		`ax2.bar(days, [5, 2, 0, 1, 8, 12, 3], alpha=0.3)`
		`fig.savefig('twin.png')`

11	Style, size and cleaning up	`plt.style.use('ggplot')`
		`plt.rcParams['figure.figsize'] = (6, 4)`
		`plt.close('all')`

Note. plt.* draws on the current figure (quick scripts); fig, ax = plt.subplots() gives explicit objects (several panels). Call savefig before show().

Module 3: Scala Data Structures and Object-Oriented Programming (12 hours)

3.1 Functions: the expanded function format

Syllabus: Module 3: Expanded Function Format. 17 rows, in running order. No extra libraries.

T.No	Task Desc	Program code

1	Entry point: statements below go inside main	`object Experiment {`
		`  def main(args: Array[String]): Unit = {`
		`  // statements go here`
		`  }`
		`}`

2	Full form: parameters, result type, block body	`def add(a: Int, b: Int): Int = {`
		`  a + b`
		`}`
		`println(add(2, 3))`

3	Short form: one expression, inferred result type	`def square(x: Int) = x * x` `println(square(7))`
4	A function with no useful result (Unit)	`def greet(name: String): Unit = println(s"Hello, $name")` `greet("VIT")`

5	Default and named arguments	`def area(w: Double = 1.0, h: Double = 1.0): Double = w * h`
		`println(area())`
		`println(area(h = 5, w = 2))`

6	Variable number of arguments	`def total(nums: Int*): Int = nums.sum`
		`println(total(1, 2, 3))`
		`println(total(List(4, 5, 6): _*))`

7	Function values (literals) and their types	`val double: Int => Int = x => x * 2`
		`val add3 = (a: Int, b: Int, c: Int) => a + b + c`
		`println(double(21))`
		`println(add3(1, 2, 3))`

8	Placeholder syntax	`val nums = List(1, 2, 3, 4)`
		`println(nums.map(_ * 10))`
		`println(nums.filter(_ % 2 == 0))`
		`println(nums.reduce(_ + _))`

9	Higher-order function: takes a function	`def applyTwice(f: Int => Int, x: Int): Int = f(f(x))`
		`println(applyTwice(_ + 3, 10))`
		`println(applyTwice(double, 5))`

10	A function that returns a function (closure)	`def multiplier(factor: Int): Int => Int = x => x * factor`
		`val triple = multiplier(3)`
		`println(triple(14))`

11	Currying: several parameter lists	`def addC(a: Int)(b: Int): Int = a + b`
		`val add10 = addC(10) _`
		`println(addC(1)(2))`
		`println(add10(5))`

12	Tail recursion	`import scala.annotation.tailrec`

13	Recursive factorial that cannot overflow the stack	`@tailrec`
		`def factorial(n: Int, acc: BigInt = 1): BigInt =`
		`  if (n <= 1) acc else factorial(n - 1, acc * n)`
		`println(factorial(20))`

14	By-name parameter: evaluated every time it is used	`def twice(block: => Unit): Unit = { block; block }` `twice(println("run"))`

15	Generic function	`def firstOf[T](items: List[T]): Option[T] = items.headOption`
		`println(firstOf(List("a", "b")))`
		`println(firstOf(List.empty[Int]))`

16	Nested function and a tuple result	`def stats(xs: List[Double]): (Double, Double) = {`
		`  def mean(v: List[Double]) = v.sum / v.size`
		`  val m = mean(xs)`
		`  (m, math.sqrt(xs.map(x => math.pow(x - m, 2)).sum / xs.size))`
		`}`
		`val (mu, sd) = stats(List(2.0, 4.0, 4.0, 4.0, 5.0, 5.0, 7.0, 9.0))`
		`println(s"mean=$mu sd=$sd")`

17	Turn a method into a function value	`def inc(x: Int) = x + 1`
		`val f = inc _`
		`println(List(1, 2, 3).map(inc))`
		`println(f(9))`

Note. def name(parameters): ResultType = { body }. The last expression is the result, no return keyword is needed. Recursive functions must state their result type.

3.2 Variables, strings, numbers and types

Syllabus: Module 3: Variables and Strings, Numbers, Variable types. 17 rows, in running order. No extra libraries.

T.No	Task Desc	Program code

1	Entry point: statements below go inside main	`object Experiment {`
		`  def main(args: Array[String]): Unit = {`
		`  // statements go here`
		`  }`
		`}`

2	val, var and lazy val	`val pi = 3.14159`
		`var counter = 0`
		`counter += 1`
		`lazy val heavy = { println("computing..."); 42 }`
		`println(counter)`
		`println(heavy)`
		`println(heavy)`

3	Type inference and explicit types	`val a = 10`
		`val b: Double = 10`
		`val c = "text"`
		`val d = 3.5f`
		`val e: Long = 1L << 40`
		`println((a.getClass.getSimpleName, b.getClass.getSimpleName,`
		`  c.getClass.getSimpleName, d, e))`

4	Strings: length, index, case, substring	`val s = "  Data Science  "`
		`val t = s.trim`
		`println(t.length)`
		`println(t(0))`
		`println(t.toUpperCase)`
		`println(t.toLowerCase)`
		`println(t.substring(0, 4))`
		`println(t.charAt(5))`
		`println(t.reverse)`

5	Search, replace, split, count	`println(t.contains("Sci"))`
		`println(t.indexOf("Science"))`
		`println(t.startsWith("Data"))`
		`println(t.replace("Data", "Big"))`
		`println(t.split(" ").toList)`
		`println(t.count(_ == 'a'))`

6	String interpolation: s, f, raw	`val name = "Asha"`
		`val marks = 91.456`
		`println(s"$name scored ${marks + 1}")`
		`println(f"$name scored $marks%.1f")`
		`println(raw"no\tescape")`

7	Multi-line strings	`val text = """first line`
		`  |second line""".stripMargin`
		`println(text)`

8	Compare and join strings	`println("abc" == "abc")`
		`println("apple" < "banana")`
		`println("a" + "b" * 3)`
		`println(List("x", "y", "z").mkString("-"))`
		`println("b".compareTo("a"))`

9	Numeric types and their limits	`println((Byte.MaxValue, Short.MaxValue, Int.MaxValue, Long.MaxValue))`
		`println(Int.MaxValue + 1)`
		`println(Double.MaxValue > 1e300)`

10	Integer and floating-point arithmetic	`println(7 / 2)`
		`println(7 % 2)`
		`println(7 / 2.0)`
		`println(7.toDouble / 2)`
		`println(-7 / 2)`
		`println(-7 % 3)`
		`println(math.floorMod(-7, 3))`

11	Conversions	`println("42".toInt + 1)`
		`println(3.99.toInt)`
		`println(65.toChar)`
		`println('A'.toInt)`
		`println(10.toString + "!")`
		`println("3.5".toDouble * 2)`
		`println("abc".toIntOption)`

12	Math functions and rounding	`println(math.sqrt(16))`
		`println(math.pow(2, 10))`
		`println(math.abs(-5))`
		`println(math.max(3, 9))`
		`println(math.round(2.5))`
		`println(math.ceil(2.1))`
		`println(math.floor(2.9))`
		`println(f"${math.Pi}%.4f")`

13	Big numbers and exact decimals	`val big = BigInt("12345678901234567890123456789O")`
		`println(big * 2)`
		`println(BigDecimal("0.1") + BigDecimal("0.2"))`
		`println(0.1 + 0.2)`

T.No	Task Desc	Program code

14	Characters	`val ch = 'q'`
		`println(ch.isLetter)`
		`println(ch.toUpper)`
		`println(ch + 1)`
		`println((ch + 1).toChar)`

15	The type hierarchy: Any, AnyVal, AnyRef	`val things: List[Any] = List(1, "two", 3.0, true, 'c')`
		`things.foreach(t => println(t.getClass.getSimpleName))`
		`println(things(1).isInstanceOf[String])`
		`println(things(0).asInstanceOf[Int] + 1)`

16	Option instead of null	`val maybe: Option[Int] = Some(5)`
		`val none: Option[Int] = None`
		`println(maybe.getOrElse(0))`
		`println(none.getOrElse(0))`
		`println(maybe.map(_ * 2))`

17	Type alias and tuples	`type Point = (Double, Double)`
		`val p: Point = (1.5, 2.5)`
		`println(p._1 + p._2)`

Note. val cannot be reassigned, var can. Int / Int is integer division. Int.MaxValue + 1 silently wraps around to a negative number. Use BigInt / BigDecimal when exactness matters.

3.3 Operators, booleans, control flow and user input

Syllabus: Module 3: Getting user input, Operators, Booleans. 18 rows, in running order. No extra libraries.

T.No	Task Desc	Program code

1	Entry point: statements below go inside main	`object Experiment {`
		`  def main(args: Array[String]): Unit = {`
		`  // statements go here`
		`  }`
		`}`

2	Input reading and loop control imports	`import scala.io.StdIn` `import scala.util.control.Breaks._`

3	Arithmetic operators	`val a = 17`
		`val b = 5`
		`println(a + b)`
		`println(a - b)`
		`println(a * b)`
		`println(a / b)`
		`println(a % b)`

4	Relational operators	`println((a > b, a < b, a >= 17, a <= 5, a == b, a != b))`

5	Logical operators and short-circuit evaluation	`val t = true`
		`val f = false`
		`println((t && f, t || f, !t, t ^ f))`
		`def boom(): Boolean = { println("evaluated"); true }`
		`println(f && boom())`
		`println(t || boom())`

6	Bitwise operators	`println((12 & 10, 12 | 10, 12 ^ 10, ~12, 1 << 4, -16 >> 2, -16 >>> 28))`

7	Compound assignment	`var x = 10`
		`x += 5`
		`x -= 3`
		`x *= 2`
		`x /= 4`
		`x %= 4`
		`println(x)`

8	Operators are methods	`println(5.+(3))`
		`println(5 max 9)`
		`println(List(1, 2) ++ List(3))`
		`println(1 to 3)`

9	Precedence	`println(2 + 3 * 4)`
		`println((2 + 3) * 4)`
		`println(10 - 4 - 3)`
		`println(2 + 3 > 4 && 1 < 2)`

10	Boolean values in collections	`val isEven = (n: Int) => n % 2 == 0`
		`println(isEven(4))`
		`val nums = List(1, 2, 3, 4)`
		`println((nums.count(isEven), nums.exists(_ > 3), nums.forall(_ > 0)))`

11	if / else is an expression	`val n = 15`
		`val size = if (n < 10) "small" else if (n < 20) "medium" else "large"`
		`println(size)`

T.No	Task Desc	Program code

12	match with guards and types	`def describe(x: Any): String = x match {`
		`  case 0 => "zero"`
		`  case i: Int if i < 0 => "negative int"`
		`  case _: Int => "positive int"`
		`  case s: String => s"text of length ${s.length}"`
		`  case (p, q) => s"pair $p,$q"`
		`  case _ => "something else"`
		`}`
		`List(0, -4, 8, "hey", (1, 2), 2.5).foreach(v => println(describe(v)))`

13	for loops: ranges, until, by, nested	`for (i <- 1 to 3) print(s"$i ")`
		`println()`
		`for (i <- 1 until 10 by 3) print(s"$i ")`
		`println()`
		`for (i <- 1 to 3; j <- 1 to 2) print(s"($i,$j) ")`
		`println()`

14	for with a guard and yield	`val squares = for (i <- 1 to 10 if i % 2 == 0) yield i * i` `println(squares)`

15	while and do-while	`var k = 3`
		`while (k > 0) { print(k + " "); k -= 1 }`
		`println()`
		`var m = 0`
		`do { m += 1 } while (m < 5)`
		`println(m)`

16	break out of a loop	`breakable {`
		`  for (i <- 1 to 10) {`
		`  if (i == 4) break()`
		`  print(s"$i ")`
		`  }`
		`}`
		`println()`

17	Read from the keyboard	`print("Enter your name: ")`
		`val name = StdIn.readLine()`
		`print("Enter marks: ")`
		`val marks = StdIn.readInt()`
		`println(s"Hello $name, your marks are $marks")`

18	Read several numbers from one line	`val numbers = StdIn.readLine().split(" ").map(_.toInt)` `println(numbers.sum)`

Note. Scala has no break or continue keyword: use breakable { ... break() } or filter the loop with a guard. A user typing something that is not a number makes readInt throw NumberFormatException.

3.4 Arrays, HashSet and the other mutable collections; the REPL

Syllabus: Module 3: Arrays, Hash set, Scala Array-REPL. 15 rows, in running order. No extra libraries.

T.No	Task Desc	Program code

1	Entry point: statements below go inside main	`object Experiment {`
		`  def main(args: Array[String]): Unit = {`
		`  // statements go here`
		`  }`
		`}`

2	Imports	`import scala.collection.mutable` `import scala.collection.immutable.{HashSet, TreeMap, TreeSet}`

3	Mutable HashSet: unique elements, fast lookup	`val hs = mutable.HashSet(3, 1, 4, 1, 5)`
		`hs += 9`
		`hs -= 1`
		`println(hs.contains(4))`
		`println(hs.size)`
		`println(hs.toList.sorted)`

4	Set algebra on HashSets	`val s1 = mutable.HashSet(1, 2, 3)`
		`val s2 = mutable.HashSet(3, 4)`
		`println((s1 union s2).toList.sorted)`
		`println((s1 intersect s2).toList)`
		`println((s1 diff s2).toList.sorted)`
		`println(s1.subsetOf(s1 union s2))`

5	Immutable HashSet returns a new set	`val frozen = HashSet("a", "b")`
		`val more = frozen + "c"`
		`println((frozen.size, more.size))`

6	ArrayBuffer: a growable array	`val buf = mutable.ArrayBuffer(1, 2, 3)`
		`buf += 4`
		`buf ++= List(5, 6)`
		`buf.insert(0, 0)`
		`buf.remove(1)`
		`println(buf)`

T.No	Task Desc	Program code

7	ListBuffer	`val lb = mutable.ListBuffer[String]()`
		`lb += "x"`
		`lb += "y"`
		`println(lb.toList)`

8	Mutable queue and stack	`val q = mutable.Queue(1, 2, 3)`
		`q.enqueue(4)`
		`println(q.dequeue())`
		`val st = mutable.Stack(1, 2, 3)`
		`st.push(9)`
		`println(st.pop())`
		`println(q)`
		`println(st)`

9	Mutable Map	`val stock = mutable.Map("apple" -> 10)`
		`stock("pear") = 5`
		`stock += ("fig" -> 2)`
		`stock.getOrElseUpdate("kiwi", 7)`
		`stock -= "fig"`
		`println(stock.toList.sortBy(_._1))`

10	Sorted collections	`val tm = TreeMap("b" -> 2, "a" -> 1)`
		`val ts = TreeSet(5, 1, 3)`
		`println(tm)`
		`println(ts)`

11	Array: create, update, sort	`val arr = Array(5, 3, 8, 1)`
		`arr(0) = 50`
		`println(arr.mkString(", "))`
		`println(arr.length)`
		`println(arr.sorted.mkString(","))`
		`println(arr.sortWith(_ > _).mkString(","))`
		`println(arr.reverse.toList)`

12	More array operations	`println(arr.indexOf(8))`
		`println(arr.slice(1, 3).toList)`
		`println(arr.max)`
		`println(arr.contains(3))`
		`println(arr.zip(Array("a", "b", "c", "d")).toList)`
		`println(Array.fill(3)(0).toList)`
		`println(Array.tabulate(4)(i => i * i).toList)`
		`println(Array.range(0, 10, 3).toList)`

13	Two-dimensional arrays	`val grid = Array.ofDim[Int](2, 3)`
		`grid(1)(2) = 7`
		`grid.foreach(row => println(row.mkString(" ")))`
		`val table = Array(Array(1, 2), Array(3, 4))`
		`println(table.map(_.sum).toList)`
		`println(table.flatten.toList)`

14	Arrays compare by reference	`val x = Array(1, 2)`
		`val y = Array(1, 2)`
		`println(x == y)`
		`println(x.sameElements(y))`
		`println(x.toList == y.toList)`

15	The Scala REPL: type an expression, read the answer	`scala> val a = Array(1, 2, 3)`
		`val a: Array[Int] = Array(1, 2, 3)`
		
		`scala> a.map(_ * 2)`
		`val res0: Array[Int] = Array(2, 4, 6)`
		
		`scala> a(0) = 9`
		
		`scala> a`
		`val res2: Array[Int] = Array(9, 2, 3)`
		
		`scala> a.length`
		`val res3: Int = 3`
		
		`scala> a :+ 4`
		`val res4: Array[Int] = Array(9, 2, 3, 4)`
		
		`scala> val m = Array.ofDim[Int](2, 2)`
		`val m: Array[Array[Int]] = Array(Array(0, 0), Array(0, 0))`
		
		`scala> m(1)(1) = 5`
		
		`scala> m`
		`val res6: Array[Array[Int]] = Array(Array(0, 0), Array(0, 5))`
		
		`scala> a.sum`
		`val res7: Int = 14`

Note. Start the REPL by typing `scala` (or `scala-cli repl`) in a terminal; `:quit` leaves it. The REPL names unnamed results res0, res1, ... and an assignment such as a(0) = 9 also uses up a number even though it prints nothing. The transcript was produced by running exactly those lines in the Scala 2.13 REPL engine.

3.5 Classes and objects: constructors, overloading, this, singleton, companion, case classes, access modifiers

Syllabus: Module 3: Scala Object and Class, Anonymous object, Singleton, Companion Object, Case Classes, Constructors, Method Overloading, This Keyword, Access Modifiers. 28 rows, in running order. No extra libraries.

T.No	Task Desc	Program code

1	Entry point: statements below go inside main	`object Experiment {`
		`  def main(args: Array[String]): Unit = {`
		`  // statements go here`
		`  }`
		`}`

2	A class with constructor parameters, fields and methods	`class Student(val name: String, var marks: Int) {`
		`  private val college = "VIT"`
		`  def grade: String = if (marks >= 90) "S" else if (marks >= 75) "A" else "B"`
		`  def curve(extra: Int): Unit = { marks += extra }`
		`  override def toString: String = s"$name ($marks, $grade) of $college"`
		`}`

3	Create objects and use them	`val s = new Student("Asha", 80)`
		`println(s)`
		`s.curve(12)`
		`println(s.grade)`
		`s.marks = 70`
		`println(s.marks)`
		`println(s.name)`

4	An anonymous object: created and used once	`println(new Student("Anon", 60).grade)`

5	Parameters without val / var stay private	`class Counter(start: Int) {`
		`  private var count = start`
		`  def next(): Int = { count += 1; count }`
		`}`

6	Use it	`val counter = new Counter(10)`
		`println(counter.next())`
		`println(counter.next())`

7	Auxiliary constructors and the this keyword	`class Rectangle(val width: Double, val height: Double) {`
		`  def this(side: Double) = this(side, side)`
		`  def this() = this(1.0)`
		`  def area: Double = width * height`
		`  def scale(k: Double): Rectangle = new Rectangle(width * k, height * k)`
		`  def biggerThan(other: Rectangle): Boolean = this.area > other.area`
		`}`

8	Call the three constructors	`println(new Rectangle(2, 3).area)`
		`println(new Rectangle(4).area)`
		`println(new Rectangle().area)`
		`println(new Rectangle(2, 3).scale(2).area)`
		`println(new Rectangle(5).biggerThan(new Rectangle(2, 3)))`

9	Return this to chain method calls	`class Builder {`
		`  private val parts = scala.collection.mutable.ListBuffer[String]()`
		`  def add(p: String): this.type = { parts += p; this }`
		`  def build: String = parts.mkString(" ")`
		`}`

10	Chain the calls	`println(new Builder().add("data").add("science").build)`

11	Method overloading: same name, different parameters	`class Printer {`
		`  def show(x: Int): String = s"int $x"`
		`  def show(x: String): String = s"string $x"`
		`  def show(x: Int, y: Int): String = s"two ints ${x + y}"`
		`}`

12	Call the overloaded methods	`val p = new Printer`
		`println(p.show(5))`
		`println(p.show("hi"))`
		`println(p.show(2, 3))`

13	Singleton object: exactly one instance, no new	`object AppConfig {`
		`  val appName = "PDS"`
		`  private var visits = 0`
		`  def visit(): Int = { visits += 1; visits }`
		`}`

14	Use the singleton	`println(AppConfig.appName)`
		`AppConfig.visit()`
		`println(AppConfig.visit())`

T.No	Task Desc	Program code

15	Companion object: factory apply, shared constants, private constructor	`class Account private (val id: Int, var balance: Double)`
		
		`object Account {`
		`  private var nextId = 1000`
		`  val MinBalance = 100.0`
		`  def apply(initial: Double): Account = {`
		`  require(initial >= MinBalance,`
		`  s"opening balance must be at least $MinBalance")`
		`  nextId += 1`
		`  new Account(nextId, initial)`
		`  }`
		`}`

16	Use the companion	`val acc = Account(500)`
		`println((acc.id, acc.balance))`
		`println(Account.MinBalance)`
		`try Account(5) catch { case e: IllegalArgumentException =>`
		`  println(e.getMessage) }`

17	unapply: make an object usable in pattern matching	`object Email {`
		`  def unapply(s: String): Option[(String, String)] = s.split("@") match {`
		`  case Array(user, domain) => Some((user, domain))`
		`  case _ => None`
		`  }`
		`}`

18	Match with the extractor	`"asha@vit.ac.in" match {`
		`  case Email(u, d) => println(s"user=$u domain=$d")`
		`  case _ => println("not an email")`
		`}`

19	A trait implemented by an anonymous class	`trait Greeter { def greet(name: String): String }`

20	Anonymous classes and the shorter lambda form	`val formal = new Greeter { def greet(name: String) = s"Good day, $name" }`
		`val shout: Greeter = name => name.toUpperCase + "!"`
		`println(formal.greet("Rao"))`
		`println(shout.greet("hey"))`

21	Case class: equality, copy, pattern matching come for free	`case class Employee(name: String, dept: String, salary: Double)`

22	Use the case class	`val e1 = Employee("Asha", "IT", 50000)`
		`val e2 = e1.copy(salary = 55000)`
		`println(e1)`
		`println(e1 == Employee("Asha", "IT", 50000))`
		`println(e2.salary)`
		`e1 match {`
		`  case Employee(n, "IT", s) if s > 40000 =>`
		`  println(s"$n is a well-paid IT employee")`
		`  case _ => println("other")`
		`}`

23	Case class with its own companion method	`case class Temperature(celsius: Double) {`
		`  def fahrenheit: Double = celsius * 9 / 5 + 32`
		`}`
		
		`object Temperature {`
		`  def fromFahrenheit(f: Double): Temperature = Temperature((f - 32) * 5 / 9)`
		`}`

24	Use it	`println(Temperature.fromFahrenheit(212))` `println(Temperature(100).fahrenheit)`

25	Access modifiers: private, protected, public	`class Vault {`
		`  private val secret = 42  // inside Vault only (other Vault`
		`  objects included)`
		`  protected val hint = "starts with 4"  // Vault and its subclasses`
		`  val label = "public"  // everywhere (the default)`
		`  def reveal(other: Vault): Int = other.secret`
		`}`
		
		`class SubVault extends Vault {`
		`  def showHint: String = hint`
		`}`

26	Use them	`val v = new Vault`
		`println(v.label)`
		`println(v.reveal(new Vault))`
		`println(new SubVault().showHint)`

27	Write equals and hashCode yourself for an ordinary class	`class Point2(val x: Int, val y: Int) {`
		`  override def equals(o: Any): Boolean = o match {`
		`  case p: Point2 => p.x == x && p.y == y`
		`  case _ => false`
		`  }`
		`  override def hashCode: Int = (x, y).hashCode`
		`}`

28	Compare them	`println(new Point2(1, 2) == new Point2(1, 2))` `println(Set(new Point2(1, 2), new Point2(1, 2)).size)`

Note. A class parameter marked val / var becomes a public field, an unmarked one stays private. Only a case class gets apply, copy, equals, hashCode, toString and unapply automatically; a plain class needs new. Trying to read a private member from outside does not compile (value secret in class Vault cannot be accessed).

3.6 Inheritance, overriding, final, abstract classes, traits and mixins

Syllabus: Module 3: Inheritance, Field Overriding, Final, Abstract Class, Trait, Trait Mixins. 24 rows, in running order. No extra libraries.

T.No	Task Desc	Program code

1	Entry point: statements below go inside main	`object Experiment {`
		`  def main(args: Array[String]): Unit = {`
		`  // statements go here`
		`  }`
		`}`

2	extends and override; super calls the parent version	`class Animal(val name: String) {`
		`  def sound: String = "..."`
		`  def describe: String = s"$name says $sound"`
		`}`
		
		`class Dog(name: String) extends Animal(name) {`
		`  override def sound: String = "Woof"`
		`}`
		
		`class Puppy(name: String) extends Dog(name) {`
		`  override def sound: String = super.sound + " (squeaky)"`
		`}`

3	Polymorphism: the subclass method runs	`val animals: List[Animal] = List(new Animal("Generic"), new Dog("Rex"), new Puppy("Bit"))` `animals.foreach(a => println(a.describe))`

4	Type tests and pattern matching on classes	`animals.foreach {`
		`  case d: Dog => println(s"${d.name} is a Dog")`
		`  case a => println(s"${a.name} is some Animal")`
		`}`
		`println(animals(1).isInstanceOf[Dog])`
		`println(animals.map(_.getClass.getSimpleName))`

5	Field overriding: override a val	`class Vehicle {`
		`  val wheels: Int = 4`
		`  def info: String = s"$wheels wheels"`
		`}`
		
		`class Bike extends Vehicle {`
		`  override val wheels: Int = 2`
		`}`

6	See the overridden field	`println(new Vehicle().info)` `println(new Bike().info)`

7	Trap: a parent constructor reads the field before the child sets it	`class Base {`
		`  val size: Int = 10`
		`  val doubled: Int = size * 2`
		`}`
		
		`class Derived extends Base {`
		`  override val size: Int = 100`
		`}`

8	The surprising result	`val d = new Derived` `println((d.size, d.doubled))`

9	final: no overriding and no further subclasses	`class Parent {`
		`  final def id: Int = 1`
		`}`
		
		`final class Leaf extends Parent`

10	Use the final members	`println(new Leaf().id)`

11	Abstract class: abstract members plus concrete ones	`abstract class Shape {`
		`  def area: Double  // abstract: no body`
		`  val sides: Int  // abstract value`
		`  def describe: String = f"$sides sides, area $area%.2f"`
		`}`
		
		`class Square(side: Double) extends Shape {`
		`  val sides = 4`
		`  def area: Double = side * side`
		`}`
		
		`class Circle(r: Double) extends Shape {`
		`  val sides = 0`
		`  def area: Double = math.Pi * r * r`
		`}`

12	Use the abstract class	`List(new Square(3), new Circle(1)).foreach(s => println(s.describe))`

T.No	Task Desc	Program code

13	Traits: reusable behaviour, with abstract and concrete members	`trait Logger {`
		`  def log(msg: String): Unit = println(s"[LOG] $msg")`
		`}`
		
		`trait Auditable {`
		`  def auditId: String`
		`}`
		
		`class Service(val auditId: String) extends Logger with Auditable {`
		`  def run(): Unit = log(s"service $auditId started")`
		`}`

14	Use the mixed-in trait	`new Service("S-1").run()`

15	Mix traits in when the object is created	`class Basic`
		`trait Walks { def move = "walks" }`
		`trait Swims { def swim = "swims" }`

16	Create a mixed object	`val duck = new Basic with Walks with Swims` `println(duck.move + " and " + duck.swim)`

17	Stackable traits: the order of mixing changes the result	`abstract class Text {`
		`  def render: String`
		`}`
		
		`class Plain extends Text {`
		`  def render = "hello"`
		`}`
		
		`trait Reverse extends Text {`
		`  abstract override def render: String = super.render.reverse`
		`}`
		
		`trait Exclaim extends Text {`
		`  abstract override def render: String = super.render + "!"`
		`}`

18	Compare the two orders	`println((new Plain with Reverse with Exclaim).render)` `println((new Plain with Exclaim with Reverse).render)`

19	Sealed trait with case classes: the compiler checks the match is complete	`sealed trait Figure`
		`case class Circ(r: Double) extends Figure`
		`case class Rect(w: Double, h: Double) extends Figure`
		`case object Dot extends Figure`

20	Match on the sealed hierarchy	`def areaOf(f: Figure): Double = f match {`
		`  case Circ(r)    => math.Pi * r * r`
		`  case Rect(w, h) => w * h`
		`  case Dot        => 0.0`
		`}`
		`println(List(Circ(1), Rect(2, 3), Dot).map(areaOf).map(a => f"$a%.2f"))`

21	Constructors run from the top of the hierarchy down	`class A { println("A constructor") }`
		`class B extends A { println("B constructor") }`
		`class C extends B { println("C constructor") }`

22	Create a C	`new C`

23	Ordered: compare your own objects	`class Version(val major: Int, val minor: Int) extends Ordered[Version] {`
		`  def compare(that: Version): Int = if (major != that.major) major -`
		`      that.major else minor - that.minor`
		`  override def toString = s"$major.$minor"`
		`}`

24	Sort and compare versions	`println(List(new Version(2, 1), new Version(1, 9), new Version(2, 0)).sorted)` `println(new Version(1, 0) < new Version(1, 1))`

Note. Abstract class: can have constructor parameters, one parent only. Trait: no constructor parameters, many can be mixed in (linearisation decides the order). Trying to override a final member or to extend a final class does not compile (`cannot override final member`, `illegal inheritance from final class`).

Module 4: Scala Libraries and Spark Basics (8 hours)

4.1 Exception handling in Scala

Syllabus: Module 4: Exception Handling. 16 rows, in running order. No extra libraries.

T.No	Task Desc	Program code

1	Entry point: statements below go inside main	`object Experiment {`
		`  def main(args: Array[String]): Unit = {`
		`  // statements go here`
		`  }`
		`}`

2	Imports	`import scala.util.{Try, Success, Failure, Using}` `import scala.util.control.NonFatal`

3	try / catch / finally	`try {`
		`  val n = "abc".toInt`
		`  println(n)`
		`} catch {`
		`  case e: NumberFormatException => println(s"bad number: ${e.getMessage}")`
		`} finally {`
		`  println("cleanup always runs")`
		`}`

4	Several kinds of exception in one catch	`def risky(i: Int): String =`
		`  try {`
		`  i match {`
		`  case 0 => throw new IllegalArgumentException("zero not allowed")`
		`  case 1 =>`
		`  val a = Array(1)`
		`  a(5).toString`
		`  case 2 => (10 / (i - 2)).toString`
		`  case _ => "ok"`
		`  }`
		`  } catch {`
		`  case e: IllegalArgumentException => s"IAE: ${e.getMessage}"`
		`  case _: ArrayIndexOutOfBoundsException => "index out of range"`
		`  case _: ArithmeticException => "division by zero"`
		`  }`
		`println(List(0, 1, 2, 3).map(risky))`

5	try is an expression: it has a value	`val parsed = try "42".toInt catch { case _: NumberFormatException => -1 }` `println(parsed)`
6	A custom exception class	`class InsufficientFundsException(val needed: Double, msg: String) extends` `  Exception(msg)`

7	throw it and catch it	`def withdraw(balance: Double, amt: Double): Double = {`
		`  if (amt > balance) throw new InsufficientFundsException(amt - balance,`
		`  s"short by ${amt - balance}")`
		`  balance - amt`
		`}`
		`try withdraw(100, 250) catch {`
		`  case e: InsufficientFundsException =>`
		`  println(s"${e.getMessage} (needed ${e.needed})")`
		`}`

8	require and assert	`try { require(1 > 2, "one is not greater than two") } catch { case e:`
		`  IllegalArgumentException => println(e.getMessage) }`
		`try { assert(1 + 1 == 3, "math") } catch { case e: AssertionError =>`
		`  println(e.getMessage) }`

9	Option: a value that may be missing	`def find(id: Int): Option[String] = Map(1 -> "Asha", 2 -> "Ravi").get(id)`
		`println(find(1))`
		`println(find(9))`
		`println(find(9).getOrElse("unknown"))`
		`find(2) match {`
		`  case Some(n) => println(n)`
		`  case None => println("none")`
		`}`

10	Try: an exception as a value	`val ok = Try("12".toInt)`
		`val bad = Try("x".toInt)`
		`println(ok)`
		`println(bad.isFailure)`
		`println(bad.getOrElse(0))`
		`bad match {`
		`  case Success(v) => println(v)`
		`  case Failure(e) => println(s"failed: ${e.getClass.getSimpleName}")`
		`}`
		`println(Try(10 / 0).recover { case _: ArithmeticException => Int.MaxValue })`

11	Chain Try values with a for expression	`def parse(s: String) = Try(s.toInt)`
		`val sum = for { a <- parse("4"); b <- parse("5") } yield a + b`
		`val broken = for { a <- parse("4"); b <- parse("x") } yield a + b`
		`println(sum)`
		`println(broken.isFailure)`

T.No	Task Desc	Program code

12	Either: Left carries the error, Right the result	`def safeDiv(a: Int, b: Int): Either[String, Int] = if (b == 0) Left("divide by zero") else Right(a / b)`
		`println(safeDiv(10, 2))`
		`println(safeDiv(1, 0))`
		`println(safeDiv(10, 2).map(_ + 1))`
		`safeDiv(1, 0) match {`
		`  case Left(err) => println(s"error: $err")`
		`  case Right(v) => println(v)`
		`}`

13	NonFatal: catch ordinary errors, not JVM-fatal ones	`try { sys.error("boom") } catch { case NonFatal(e) => println(s"handled: ${e.getMessage}") }`
14	Using: close a resource automatically	`val first = Using(new java.io.StringReader("data")) { r => r.read().toChar }` `println(first)`

15	Wrap an exception and keep its cause	`try {`
		`  try { "x".toInt } catch { case e: NumberFormatException => throw new RuntimeException("config invalid", e) }`
		`} catch {`
		`  case e: RuntimeException =>`
		`  println(s"${e.getMessage} <- ${e.getCause.getClass.getSimpleName}")`
		`}`

16	Inspect an exception	`try { 1 / 0 } catch { case e: Exception => println(e);` `  println(e.getStackTrace.nonEmpty) }`

Note. Scala has no checked exceptions: nothing forces you to catch. Prefer Option, Try or Either for expected failures and keep try / catch for unexpected ones. The catch block is a partial function made of case clauses, so put specific exceptions before general ones.

4.2 Apache Spark: architecture, RDDs, DataFrames, SQL, MLlib, streaming, GraphX

Syllabus: Module 4: Apache Spark Architecture, Spark Big Data, Apache Spark Components. 25 rows, in running order. Libraries: org.apache.spark::spark-sql:3.5.1, org.apache.spark::spark-mllib:3.5.1, org.apache.spark::spark-graphx:3.5.1.

T.No	Task Desc	Program code

1	Entry point: statements below go inside main	`object Experiment {`
		`  def main(args: Array[String]): Unit = {`
		`  // statements go here`
		`  }`
		`}`

2	Imports	`import org.apache.spark.sql.SparkSession`
		`import org.apache.spark.sql.functions._`
		`import org.apache.spark.sql.expressions.Window`
		`import java.nio.file.{Files, Paths}`

3	The data type used for DataFrames and Datasets	`case class Emp(name: String, dept: String, salary: Double, age: Int)`

4	Start Spark in local mode: this program is the driver	`val spark =`
		`  SparkSession.builder().appName("pds").master("local[*]").getOrCreate()`
		`val sc = spark.sparkContext`
		`sc.setLogLevel("ERROR")`
		`import spark.implicits._`
		`println(spark.version)`

5	Architecture facts: master, parallelism, partitions	`println(sc.master)`
		`println(sc.defaultParallelism > 0)`
		`println(sc.parallelize(1 to 100, 4).getNumPartitions)`

6	RDD: transformations are lazy, actions run the job	`val nums = sc.parallelize(1 to 10)`
		`val evensSquared = nums.filter(_ % 2 == 0).map(x => x * x)`
		`println(evensSquared.collect().toList)`
		`println(evensSquared.count())`
		`println(nums.reduce(_ + _))`

7	Word count with reduceByKey	`val lines = sc.parallelize(Seq("spark is fast", "spark is lazy",`
		`  "scala runs spark"))`
		`val counts = lines.flatMap(_.split(" ")).map(w => (w, 1)).reduceByKey(_ + _)`
		`println(counts.collect().sortBy(r => (-r._2, r._1)).toList)`

8	The lineage (DAG) behind an RDD	`println(counts.toDebugString)`

9	Pair RDD join and caching	`val names = sc.parallelize(Seq((1, "Asha"), (2, "Ravi")))`
		`val marks = sc.parallelize(Seq((1, 90), (2, 75), (3, 60)))`
		`println(names.join(marks).collect().sortBy(_._1).toList)`
		`val cached = nums.map(_ * 2).cache()`
		`println(cached.sum())`

10	Shared variables: broadcast and accumulator	`val lookup = sc.broadcast(Map(1 -> "IT", 2 -> "HR"))`
		`val acc = sc.longAccumulator("evens")`
		`sc.parallelize(1 to 10).foreach(x => if (x % 2 == 0) acc.add(1))`
		`println(acc.value)`
		`println(sc.parallelize(Seq(1, 2, 1)).map(lookup.value).collect().toList)`

T.No	Task Desc	Program code

11	DataFrame from case classes	`val emp = Seq(Emp("Asha", "IT", 90000, 28), Emp("Ravi", "HR", 60000, 35),`
		`  Emp("Meena", "IT", 110000, 41), Emp("Kabir", "HR", 65000,`
		`  30)).toDF()`
		`emp.printSchema()`
		`emp.show()`

12	select, filter, withColumn, orderBy	`emp.select($"name", $"salary")`
		`  .filter($"salary" > 62000)`
		`  .withColumn("bonus", $"salary" * 0.1)`
		`  .orderBy($"salary".desc)`
		`  .show()`

13	groupBy and aggregations	`emp.groupBy("dept")`
		`  .agg(count("*").as("people"), avg("salary").as("avg_salary"),`
		`  max("age").as("oldest"))`
		`  .orderBy("dept").show()`

14	Spark SQL on a temporary view	`emp.createOrReplaceTempView("employees")`
		`spark.sql("SELECT dept, ROUND(AVG(salary), 1) AS avg_salary FROM employees WHER`
		`  E age > 29 GROUP BY dept ORDER BY dept").show()`

15	Join two DataFrames	`val depts = Seq(("IT", "Bengaluru"), ("HR", "Pune")).toDF("dept", "city")`
		`emp.join(depts, Seq("dept"), "inner").select("name", "dept",`
		`  "city").orderBy("name").show()`

16	Window function: rank inside each department	`val w = Window.partitionBy("dept").orderBy($"salary".desc)`
		`emp.withColumn("rank", rank().over(w)).select("name", "dept",`
		`  "rank").orderBy("dept", "rank").show()`

17	Typed Dataset	`val ds = emp.as[Emp]` `println(ds.filter(_.age > 29).map(_.name).collect().toList)`
18	User-defined function	`val level = udf((s: Double) => if (s >= 100000) "high" else "normal")` `emp.withColumn("level", level($"salary")).select("name", "level").show()`

19	Read a CSV file	`Files.write(Paths.get("people.csv"), "name,age\nAsha,28\nRavi,35\n".getBytes)`
		`val people = spark.read.option("header", "true").option("inferSchema",`
		`  "true").csv("people.csv")`
		`people.show()`
		`people.printSchema()`

20	The execution plan Spark will run	`emp.filter($"age" > 29).groupBy("dept").count().explain()`

21	MLlib: K-means clustering	`import org.apache.spark.ml.feature.VectorAssembler`
		`import org.apache.spark.ml.clustering.KMeans`
		`val pts = Seq((1.0, 1.0), (1.2, 0.8), (0.9, 1.1), (8.0, 8.0), (8.2, 7.9), (7.8,`
		`  8.1)).toDF("x", "y")`
		`val feats = new VectorAssembler().setInputCols(Array("x",`
		`  "y")).setOutputCol("features").transform(pts)`
		`val model = new KMeans().setK(2).setSeed(1L).fit(feats)`
		`println(model.clusterCenters.map(c => c.toArray.map(v =>`
		`  math.round(v)).toList).sortBy(_.head).toList)`

22	MLlib: linear regression	`import org.apache.spark.ml.regression.LinearRegression`
		`val train = Seq((1.0, 3.0), (2.0, 5.0), (3.0, 7.0), (4.0, 9.0)).toDF("x",`
		`  "label")`
		`val data = new`
		`  VectorAssembler().setInputCols(Array("x")).setOutputCol("features").transfo`
		`  rm(train)`
		`val lr = new LinearRegression().fit(data)`
		`println((math.round(lr.coefficients(0)), math.round(lr.intercept)))`

23	Structured Streaming: a rate source into a memory table
	(not run: Spark writes a checkpoint, which needs winutils.exe on Windows (standard code on Linux / Colab))
		`val stream = spark.readStream.format("rate").option("rowsPerSecond", 5).load()`
		`val query =`
		`  stream.writeStream.format("memory").queryName("ticks").outputMode("append")`
		`  .start()`
		`Thread.sleep(3000)`
		`query.stop()`
		`println(spark.sql("SELECT COUNT(*) FROM ticks").first().getLong(0) > 0)`

24	GraphX: a small graph, degrees and PageRank	`import org.apache.spark.graphx.{Edge, Graph}`
		`val users = sc.parallelize(Seq((1L, "Asha"), (2L, "Ravi"), (3L, "Meena")))`
		`val follows = sc.parallelize(Seq(Edge(1L, 2L, "follows"), Edge(2L, 3L,`
		`  "follows"), Edge(3L, 2L, "follows")))`
		`val graph = Graph(users, follows)`
		`println(graph.inDegrees.collect().sortBy(_._1).toList)`
		`println(graph.pageRank(0.001).vertices.collect().sortBy(-_._2).head._1)`

25	Stop the session	`spark.stop()`

Note. Architecture: the driver (your program, holds the SparkSession) builds a DAG of transformations; the cluster manager gives it executors; executors run tasks on partitions. Nothing runs until an action (collect, count, show, write). Components: Spark Core (RDDs), Spark SQL / DataFrames, Structured Streaming, MLlib, GraphX. Run in local mode on one machine.

Module 5: Programming language selection criteria (6 hours)

5.1 Language selection: a weighted decision matrix

Syllabus: Module 5: Programming language selection criteria. 7 rows, in running order. Install: pip install pandas matplotlib.

T.No	Task Desc	Program code
1	Load packages	`import pandas as pd` `import matplotlib.pyplot as plt`

2	The criteria of the syllabus with importance weights (1 to 5)	`weights = pd.Series({`
		`  'Size of deployment (data, resource, load)': 5, 'Security': 4,`
		`  'Skill set': 4, 'Targeted platform': 3,`
		`  'Elasticity of the language': 3, 'Time to production': 4, 'Performance': 5,`
		`  'Support and community': 3,`
		`  'Purpose': 5, 'Programmer experience': 3,`
		`  'Ease of development and maintenance': 4, 'Efficiency': 4,`
		`  'Availability of an IDE': 2, 'Error checking and diagnosis': 3})`

3	Score each language per criterion (1 to 5; example opinions)	`scores = pd.DataFrame({`
		`  'Python': [3, 3, 5, 4, 4, 5, 3, 5, 5, 5, 4, 3, 5, 3],`
		`  'Scala':  [5, 4, 2, 4, 4, 3, 5, 3, 5, 3, 3, 4, 4, 5],`
		`  'Java':   [5, 5, 4, 5, 4, 3, 5, 5, 4, 3, 4, 4, 5, 4],`
		`  'R':      [2, 2, 3, 3, 3, 4, 2, 4, 3, 4, 3, 2, 4, 3]}, index=weights.index)`

4	Weighted total of every language	`totals = scores.mul(weights, axis=0).sum().sort_values(ascending=False)` `print(totals)`

5	Percentage of the best possible score and the winner	`percent = (totals / (weights.sum() * 5) * 100).round(1)`
		`print(percent.to_dict())`
		`print('Best choice:', totals.idxmax())`

6	Sensitivity: what if performance matters twice as much?	`w2 = weights.copy()`
		`w2['Performance'] = 10`
		`print(scores.mul(w2, axis=0).sum().sort_values(ascending=False).index.tolist())`

7	Show the result as a bar chart	`plt.figure(figsize=(5, 3))`
		`plt.bar(totals.index, totals.values, color='steelblue')`
		`plt.ylabel('Weighted score')`
		`plt.title('Language selection matrix')`
		`plt.tight_layout()`
		`plt.savefig('matrix.png')`

Note. Module 5 has no programming content in the syllabus; this is a way to turn the criteria list into a calculation. The scores are example opinions, change them to fit the scenario in the question.

What was verified and what was not

Verified by running

- Python: 269 of 271 rows were executed in order, group by group, each group in an empty folder (2 rows are marked 'not run'). Failures: 0.
- Scala: 8 programs (one per table) were compiled with Scala 2.13.12 and run (one Spark row is marked 'not run'). Failures: 0. The REPL transcript in 3.4 was fed to the real Scala 2.13 REPL engine and every reply matched.
- Rows were checked for running without an error. Printed values were read for the Scala programs and spot-checked for the Python ones, not compared one by one with an expected answer.
- Versions: Python 3.10; scipy 1.15.3, nltk 3.10.4, folium 0.20.0, requests 2.34.2, scikit-learn 1.7.2, pandas 2.3.3 (main environment); ChatterBot 1.2.15 with spaCy 3.8.18; Scrapy 2.19.0; xgboost 3.2.0, torch 2.14.1 (CPU), tensorflow 2.21.0, keras 3.12.4; Spark 3.5.1 on Java 17 with Scala 2.13.12.

Not verified, or only partly

- **The REST rows** ran against a small local server that copies the routes of jsonplaceholder.typicode.com, not against the live site. The displayed code uses the real URL; only the base URL differed during the check.
- **The Scrapy spider** ran on two local HTML files with the same structure as quotes.toscrape.com, so pagination and the pipeline were exercised, but the live site was not contacted.
- **The NLTK download row** was not run (it needs internet); the data was already installed. The terminal commands for a Scrapy project are shown but not run.
- **Spark** ran in local mode on one machine, not on a cluster. The Structured Streaming row is shown but was not run: Spark writes a checkpoint, which needs winutils.exe on Windows. No row makes Spark write files (DataFrame.write was left out because it was not tested). The Java 17 --add-opens options are needed for Spark 3.5 on Java 17.
- **Group 5.1** (language selection) turns the syllabus criteria into a calculation. The 1 to 5 scores are example opinions, not facts. Module 5 and Module 6 are theory in the syllabus.
- **Lab experiments 1 to 20** are not repeated here; they are in the cheat sheet. Debug and fill-in-the-blank questions were not written for these extra topics.
- Error texts and printed numbers can differ a little on other library versions.